[University – to be confirmed]

# Sentinel KMS: Governed Cybersecurity Knowledge Management with Retrieval-Augmented Generation

[Author's full name – to be confirmed]

[Master's programme – to be confirmed]

[Submission month and year – to be confirmed]

Supervisor-review dissertation draft

[Department – to be confirmed]

# Sentinel KMS: Governed Cybersecurity Knowledge Management with Retrieval-Augmented Generation

[Author's full name – to be confirmed]

[Master's programme – to be confirmed]

Supervisors:
[Supervisor(s), qualifications and affiliation – to be confirmed]

[University – to be confirmed]
[Submission month and year – to be confirmed]

# Acknowledgements

[Personal acknowledgements to be written by the author – to be confirmed]

# Resumo

A presente dissertação aborda a gestão de conhecimento em cibersegurança através do desenvolvimento do Sentinel KMS, um protótipo destinado a apoiar a consulta e a reutilização de informação por analistas. O problema de investigação não se limita à geração de respostas: inclui a preservação da proveniência, a autorização de acesso, a atualização dos documentos e a possibilidade de inspecionar a evidência utilizada. A geração aumentada por recuperação de informação, ou RAG, é tratada como um componente substituível e sujeito a avaliação, e não como uma garantia de correção.

O trabalho adota uma abordagem orientada à conceção e avaliação de um artefacto. A versão principal confirmada integra ingestão durável, políticas de acesso, publicação controlada, recuperação híbrida e referências a passagens de revisões específicas. Um repositório de asserções permite distinguir relações associadas a fontes de inferências declaradas; não constitui, porém, um mecanismo de recuperação por grafo. Alterações locais posteriores acrescentam apoio à consulta de planos de resposta e melhoram a interface, mas são distinguidas da versão confirmada. O assistente tem uma finalidade consultiva e não executa ações de resposta a incidentes.

A dissertação analisa a continuidade dos controlos entre ingestão, publicação, recuperação, armazenamento das respostas e inspeção posterior da evidência. A verificação inclui testes Java e da interface, percursos automatizados com respostas simuladas e verificações de relatórios. Esses resultados não medem a qualidade das respostas nem os benefícios para analistas. A comparação controlada entre condições de pesquisa e geração permanece por realizar; um estudo com participantes é proposto como trabalho futuro. A contribuição atual é a análise da integração e dos limites do sistema, sem alegar eficácia operacional ainda não demonstrada.

**Palavras-chave:** gestão de conhecimento; cibersegurança; geração aumentada por recuperação; proveniência; controlo de acesso; avaliação.

# Abstract

This dissertation investigates cybersecurity knowledge management through the development of Sentinel KMS, an advisory prototype for retrieving and reusing security information. The research problem extends beyond generating answers: it concerns the preservation of provenance, enforcement of access permissions, management of changing documents, and inspection of the evidence used in an answer. Retrieval-augmented generation (RAG) is treated as a replaceable component to be evaluated rather than as a guarantee of correctness.

The work follows an artefact-oriented design and evaluation approach. The inspected committed system integrates durable ingestion, knowledge-access policies, controlled publication, hybrid retrieval, and references to passages in specific document revisions. A minimal graph assertion store distinguishes claims associated with sources from declared inferences; it does not provide graph retrieval. Later local changes introduce response-playbook assistance and a revised interface and are distinguished from the committed version. The assistant supports human judgement rather than executing incident-response actions.

The dissertation examines control continuity across ingestion, publication, retrieval, answer storage, and later evidence inspection. Verification comprises Java and frontend tests, scripted browser interactions with mocked responses, and report-validation checks. Neither those checks nor synthetic fixtures establish answer quality or analyst benefit. Controlled comparisons of search and generation conditions remain to be conducted; a participant study is proposed as future work. The contribution is an implementation and boundary analysis of the system, not a claim of proven operational improvement.



**Keywords:** knowledge management; cybersecurity; retrieval-augmented generation; provenance; access control; evaluation.

# Contents

**Acknowledgements** i

**Resumo** iii

**Abstract** v

**List of Figures** xi

**List of Tables** xiii

**List of Acronyms** xv

**1 Introduction** 1
- 1.1 Motivation and context . . . 1
  - 1.1.1 From information availability to usable knowledge . . . 1
  - 1.1.2 Retrieval as an assistive component . . . 2
- 1.2 Problem statement . . . 2
- 1.3 Objectives and research questions . . . 3
- 1.4 Research methodology . . . 4
  - 1.4.1 Literature-informed design . . . 4
  - 1.4.2 Artefact inspection and software verification . . . 5
  - 1.4.3 Evaluation as a sequence of evidence levels . . . 5
- 1.5 Contributions and scope boundaries . . . 5
- 1.6 Dissertation structure . . . 6

**2 State of the Art** 7
- 2.1 Review approach and evidence boundaries . . . 7
  - 2.1.1 Inputs and scope . . . 7
  - 2.1.2 Levels of support . . . 8
- 2.2 Knowledge management and cybersecurity knowledge sharing . . . 8
  - 2.2.1 A KMS is more than a question-answering interface . . . 8
  - 2.2.2 Trust and organisational context in CTI . . . 9
- 2.3 Ontologies and cybersecurity knowledge graphs . . . 9
  - 2.3.1 Semantic organisation and interoperability . . . 9
  - 2.3.2 Dynamic knowledge and source revision . . . 10
  - 2.3.3 Graph quality, inferred edges, and chunking . . . 10

2.4 RAG and language models for cybersecurity work . . . 11
2.4.1 Relevant systems and their different tasks . . . 11
2.4.2 Counterevidence and baseline selection . . . 11
2.4.3 Local deployment and the unit of comparison . . . 12
2.5 Evaluating retrieval, generation, and human outcomes . . . 13
2.5.1 Component-level evaluation . . . 13
2.5.2 Abstention, conflicting evidence, and negative cases . . . 13
2.5.3 Human outcomes and the limits of automation . . . 14
2.6 Security and governance of retrieved knowledge . . . 14
2.7 Synthesis and research direction . . . 15

**3 Sentinel KMS: Design and Implementation 17**
3.1 Committed implementation and working-tree interface . . . 17
3.2 Scope and implementation requirements . . . 18
3.3 Runtime architecture and deployment boundaries . . . 19
3.4 Governance and knowledge lifecycle . . . 20
3.4.1 Identity, classification, and authorization . . . 20
3.4.2 Publication as a controlled state transition . . . 21
3.5 Immutable evidence and provenance . . . 22
3.6 Durable ingestion and source integration . . . 23
3.6.1 Admission, retention, and processing . . . 23
3.6.2 Concurrency and connector boundaries . . . 24
3.7 Access-aware hybrid retrieval and answer generation . . . 24
3.7.1 Candidate selection and ranking . . . 24
3.7.2 Grounding before generation . . . 25
3.8 Citations, presentation, and analyst inspection . . . 26
3.9 Graph assertions and operational audit . . . 29
3.10 Limits of the implementation account . . . 29

**4 Evaluation Design and Current Evidence 31**
4.1 Purpose and evidential boundary . . . 31
4.2 Available artifacts and executed checks . . . 32
4.2.1 Datasets and evaluation mechanisms . . . 32
4.2.2 What the offline run establishes . . . 32
4.2.3 Executed software verification and scripted walkthrough . . . 33
4.3 Prospective corpus and comparison design . . . 34
4.4 Metrics and operational definitions . . . 35
4.4.1 Retrieval quality . . . 35
4.4.2 Answer quality and safety . . . 36
4.5 Expert review and adversarial validation . . . 37
4.6 Statistical and ethical protocol . . . 38

4.7 Requirement validation and limitations . . . . . 38

**5 Conclusion** **41**
5.1 Conclusions supported by the present work . . . . . 41
5.2 Main contributions . . . . . 42
5.3 Limitations . . . . . 42
5.3.1 Literature and argument . . . . . 42
5.3.2 Implementation and deployment . . . . . 43
5.3.3 Evaluation and generalisation . . . . . 43
5.4 Future work and completion priorities . . . . . 43
5.5 Closing statement . . . . . 44

**A Implementation Evidence and Reproducibility Boundaries** **45**
A.1 How the evidence should be read . . . . . 45
A.2 Worked design scenario: publication, citation, and withdrawal . . . . . 45
A.3 Evidence identity and verification limits . . . . . 46

**B Source Evidence and Bibliographic Method** **49**
B.1 Research inputs and limits of inspection . . . . . 49
B.2 Claim-to-source examples . . . . . 49

**C Proposed Review and Analyst-Study Instruments** **51**
C.1 Scenario selection and requirement coverage . . . . . 51
C.2 Case review form . . . . . 52
C.3 Answer review form . . . . . 52
C.4 Analyst task record . . . . . 53

**References** **55**

# List of Figures

3.1 Conceptual boundaries in committed `main`. Arrows represent service interfaces, not a measured deployment or a claim that graph assertions enter answer generation. . . . . 20
3.2 Evidence continuity for a governed article, from publication through retrieval and citation to a subsequent access-checked passage view. Upload-only grounding is a separate path; the diagram does not assert that a citation proves answer faithfulness. . . . . 23
3.3 Knowledge-base collection in the modified frontend: a playbook-browsing band above one fixture advisory. The band is an uncommitted interface addition; the displayed document and count are mocked, not a measured corpus. . . . . 27
3.4 Ingestion view after the browser submitted a fixture upload. The accepted message, indexed and failed jobs, connector status, and index counters come from intercepted API responses; the fixture immediately reports "indexed" without a backend worker having processed the file. . . . . 27
3.5 Assistant reply with a fixture article citation and a distinct revision 7 passage action. The answer text and SSE frames are intercepted responses, not a measured language-model run or proof that the cited passage entails the advice. . . . . 28
3.6 Focused exact-evidence dialog with the fixture passage, revision, snapshot identity, span, and canonical-source action. The response is mocked; the browser contract does not exercise the committed server's authorization or establish source authenticity. . . . . 28

# List of Tables

1.1 Application of the DSRM activities to Sentinel KMS. Activity names follow Peffers et al.; project outputs and boundaries are specific to this dissertation. 4

2.1 Selected technical studies and their bounded implications. Reported outcomes below are abstract-level claims, not replications. . . . . 12

3.1 Source-inspected provenance. Governance, durable ingestion, immutable evidence, and graph storage already exist in committed `main`; the right column concerns additional uncommitted work. . . . . 17

3.2 Functional (F) and non-functional (N) requirements, implementation mechanisms, and their boundaries. . . . . 19

4.1 Observed engineering verification on the modified working tree, 29 September 2026; test counts are not participants or evaluated security questions. The report gate is a distinct synthetic-check layer, not part of the application-suite totals. . . . . 33

4.2 Requirement ID to implementation/test evidence and current status, linked to Chapter 3, Table 3.2. Passing tests establish selected assertions only; empirical acceptance remains pending. . . . . 40

A.1 Design-scenario checkpoints and their evidential status; these are not outcomes of an executed end-to-end case. . . . . 46

B.1 Illustrative claim-to-source boundaries in the literature argument. . . . . 49

C.1 Reader-facing scenario prompts mapped to the requirement identifiers in Table 4.2; no expert or analyst outcomes are reported. . . . . 51

# List of Acronyms

| | |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| ATT&CK | Adversarial Tactics, Techniques, and Common Knowledge |
| CI | Continuous Integration |
| CTI | Cyber Threat Intelligence |
| CVE | Common Vulnerabilities and Exposures |
| DFIR | Digital Forensics and Incident Response |
| DSRM | Design Science Research Methodology |
| HNSW | Hierarchical Navigable Small World |
| HTTP | Hypertext Transfer Protocol |
| IOC | Indicator of Compromise |
| JDBC | Java Database Connectivity |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| KEV | Known Exploited Vulnerabilities |
| KG | Knowledge Graph |
| KMS | Knowledge Management System |
| LLM | Large Language Model |
| MRR | Mean Reciprocal Rank |
| nDCG | Normalized Discounted Cumulative Gain |
| NLP | Natural Language Processing |
| NVD | National Vulnerability Database |
| PPI | Prediction-Powered Inference |
| QA | Question Answering |
| RAG | Retrieval-Augmented Generation |
| RGB | Retrieval-Augmented Generation Benchmark |
| RLS | Row-Level Security |
| RRF | Reciprocal-Rank Fusion |
| SLM | Small Language Model |
| SOC | Security Operations Centre |
| SQL | Structured Query Language |
| SSE | Server-Sent Events |
| SSRF | Server-Side Request Forgery |
| STIX | Structured Threat Information Expression |

| | |
|---|---|
| UI | User Interface |
| URI | Uniform Resource Identifier |

CHAPTER 1

# Introduction

Cybersecurity knowledge is useful only when it can be interpreted in the context of a decision. A vulnerability description, an incident report, a response procedure, and an internal policy may all be relevant to the same question, but they differ in purpose, authority, permitted audience, and period of validity. Research on cyber threat intelligence (CTI) sharing identifies both technical and non-technical obstacles to making such information usable across organisational boundaries.[1] Reviews of cybersecurity knowledge graphs examine ways to organise heterogeneous security information and represent the relations required for its interpretation.[20] These concerns motivate the present work: a knowledge-management system in which an analyst can retrieve information, inspect its provenance, and understand the limits of an AI-generated explanation.

The artefact examined is Sentinel KMS as implemented in the current committed principal branch. It supports knowledge recovery and review rather than acting autonomously on infrastructure. Its committed implementation already combines a document library, durable ingestion, document-level governance, hybrid retrieval, a conversational interface, and revision-specific evidence references. Later local changes add an incident-oriented response-plan interface and curated playbook support. These are distinguished from the committed system. This dissertation investigates the integration and the evidence available to assess it; it does not present a completed demonstration of analyst benefit.

## 1.1. Motivation and context

### 1.1.1. From information availability to usable knowledge

The existence of a searchable collection does not resolve the organisational problem of knowledge sharing. Ipe's conceptual framework examines factors influencing individual knowledge sharing within organisations; Yeboah's review organises the topic around enablers, processes, and outcomes. These works motivate attention to participation and reuse, while their application to security analysts remains a contextual interpretation rather than a demonstrated effect of the present system.[9, 26]

Consider an analyst asking whether a published mitigation applies to an internally deployed product. A useful response must do more than find text containing the product name. It must distinguish the affected version from other versions, identify whether the advice has been superseded, retain the source of the recommendation, and avoid revealing restricted internal details to an unauthorised requester. This is an illustrative

design scenario, not an observation from a completed user study. It makes explicit why correctness, temporal scope, provenance, and permissions are linked requirements.

The KMS perspective also changes what must be retained. An isolated answer is not a sufficient knowledge record if its supporting document can later change without preserving the original passage. Similarly, an entity relationship is not self-explanatory if there is no indication of whether it was copied from a source or inferred by a model. The ontology and knowledge-graph literature motivates attention to structured representation and graph quality.[11, 20] Preserving source revisions and separating sourced from inferred assertions are this project's design responses to those concerns; they are not evidence that a particular graph implementation improves analyst decisions.

### 1.1.2. Retrieval as an assistive component

In the operational definition used here, RAG combines the retrieval of passages from a reference corpus with a language model that generates an answer using those passages. The RAGAs and ARES papers describe this separation between retrieval and generation and demonstrate why evaluating the components independently is useful.[6, 18] This separation makes it possible to update an external collection and inspect the information supplied to the model. It also creates several points of failure: the relevant passage may not be retrieved, the retrieved passage may be unsuitable, or the model may misrepresent otherwise relevant evidence.

The motivation for Sentinel KMS is therefore not that RAG solves hallucination. It is that a retrieval-based interface can expose evidence and can be compared with alternatives under controlled conditions. ARES explicitly observes that the best configuration depends on the domain, corpus, and cost or latency budget.[18] This dissertation adopts the corresponding research position: whether retrieval improves the selected cybersecurity tasks is an empirical question, and a negative result remains a valid outcome.

In this implementation, human control takes concrete forms: contributors submit knowledge through lifecycle operations, authorised reviewers control publication, and analysts can inspect answer evidence. This does not mean that every generated answer has been approved by an expert. Literature recommendations inform the interpretation of these mechanisms, while the actual source definitions determine which capabilities this dissertation attributes to the artefact.

## 1.2. Problem statement

The problem addressed is how to recover and communicate cybersecurity knowledge while retaining the controls necessary to inspect, restrict, and revise that knowledge. The problem has four interacting dimensions.

First, information must remain traceable through ingestion, extraction, chunking, retrieval, and presentation. Second, the right to view information must be enforced by application and storage controls rather than by a model's willingness to obey a prompt.

Third, the generation layer must distinguish what the accessible evidence supports from what it does not establish. Fourth, evaluation must measure the effects of the complete workflow without confusing implementation tests with evidence of human or organisational benefit.

The intended users are analysts and knowledge contributors working with security documentation. The scope includes advisory questions about the knowledge base, source inspection, document contribution and governance, and the maintenance of evidence-linked assertions. It excludes autonomous incident remediation, unsupervised command execution, and claims about preventing attacks in production. These boundaries narrow both the implementation and the claims that an evaluation could justify.

### 1.3. Objectives and research questions

The overall objective is to design, implement, and establish a defensible evaluation procedure for a governed cybersecurity KMS in which RAG supports, but does not replace, human judgement. The principal research question is:

> How does the implemented Sentinel KMS integrate retrieval-augmented generation with provenance, lifecycle governance, and access control, and what can the available verification evidence establish about that integration?

This is an implementation-led question, addressed through source definitions, call paths, storage constraints, and bounded executable checks. Whether the integration improves analyst work is a subsequent empirical question, not an outcome presumed by the research question. Four subordinate questions organise the account:

RQ1. How are source revisions, passage references, authorisation, and lifecycle state associated across ingestion, retrieval, answer storage, and later evidence inspection?

RQ2. How does the current retrieval and generation path select accessible evidence, handle missing grounding, and expose citations, and which apparent semantic capabilities remain outside that path?

RQ3. Which implementation mechanisms address stale or revoked knowledge and hostile retrieved content, and what limits prevent their presence from establishing operational security?

RQ4. What do the available evaluation mechanisms and executed checks establish, and what additional evidence is needed to assess retrieval quality, answer support, and analyst outcomes?

The corresponding objectives are to document the implemented information and threat model; explain the integration of ingestion, governance, retrieval, and evidence inspection; distinguish software mechanisms from demonstrated guarantees; and assess the existing

evaluation infrastructure against the evidence still required. Controlled search-only, no-retrieval, and RAG comparisons, independent expert review, and an analyst study are completion priorities for subsequent empirical work rather than completed objectives of this implementation account.

## 1.4. Research methodology

The organisation of the research follows Design Science Research Methodology (DSRM). Peffers et al. describe six activities: problem identification and motivation, definition of solution objectives, design and development, demonstration, evaluation, and communication.[17] The methodology is suitable here because the research produces a software artefact whose design and behaviour must be related to a practical problem. Its use does not make the artefact effective by definition; evaluation remains an independent obligation.

Table 1.1: Application of the DSRM activities to Sentinel KMS. Activity names follow Peffers et al.; project outputs and boundaries are specific to this dissertation.

| Activity | Application and evidential boundary |
|---|---|
| Problem identification | Use the supplied literature handoffs and project scope to identify knowledge recovery, traceability, and governance needs; do not imply a completed field-needs study. |
| Solution objectives | Define advisory operation, evidence inspection, lifecycle-aware authorisation, and controlled comparison as requirements. |
| Design and development | Implement the web application, document pipeline, retrieval services, and governance mechanisms; inspect their definitions, interactions, and storage invariants. |
| Demonstration | Prepare bounded scenarios and executable software checks; distinguish fixtures from observed analyst behaviour. |
| Evaluation | Separate software conformance, live model comparison, expert annotation, and participant outcomes. The latter stages remain pending. |
| Communication | Present the architecture, literature limits, reproducibility records, and remaining evidence obligations in this dissertation. |

### 1.4.1. Literature-informed design

The literature input consists of two connected notes and the repository's research plan, recommendation, and claim matrix. The recommendation supplies an accepted-reference list, while the detailed KMS note supplies structured records describing ontology, knowledge-graph, sharing, and retrieval research. These materials form a bounded evidence map. They are not a newly conducted systematic review with a reproducible database query log and full-text appraisal. Chapter 2 documents this distinction and explains how the sources support design choices without overstating their certainty.

### 1.4.2. Artefact inspection and software verification

Implementation claims are tied to source definitions, their call paths, storage migrations, and test artefacts. Chapter 3 describes the inspected implementation rather than treating a README feature list as authoritative. A source-level observation establishes that code for a behaviour exists; a passing isolated test adds evidence only for the exercised behaviour and environment. Neither establishes a production deployment's security posture.

The committed principal branch contains the governed knowledge workflow described in this dissertation. Its local working tree also contains uncommitted changes to the incident-oriented interface, curated playbooks, and generation fallback. Where those later changes appear in an interface illustration, they are labelled as a local development state rather than attributed to the committed artefact. A separate reproducibility record identifies the exact revision and inspected local file hashes. The engineering checks reported in Chapter 4 exercised that local working tree; they are not retroactively presented as test results for the unmodified commit.

### 1.4.3. Evaluation as a sequence of evidence levels

The evaluation is deliberately staged. Structural checks establish whether a report or citation has the expected format. Deterministic tests establish whether a software boundary behaves as specified for selected cases. Provider-backed runs measure actual retrieval or generation behaviour under a declared configuration. Expert annotation assesses support and correctness in context. A participant study assesses consequences for the intended users. Each stage answers a different question, and evidence cannot be promoted from one stage to another merely because a test suite is green.

Chapter 4 defines the cases, metrics, baselines, review procedure, and limitations required for that progression. No participant count, effect size, significance result, or improvement in response time is invented to fill the evaluation chapter.

## 1.5. Contributions and scope boundaries

The first contribution is an implementation account of a governed advisory workflow: durable ingestion, controlled publication, access-aware hybrid retrieval, and answer evidence resolution. The second is an explicit representation and boundary analysis separating mutable articles, immutable revisions and passages, retrieval derivatives, graph assertions, and persisted citations. The third is a layered evaluation account linking existing fixtures, runners, and bounded report-validator checks to a prospective comparison and review protocol. These are integration, analytical, and methodological contributions, not a new retrieval algorithm or demonstrated improvements in live security outcomes.

The implementation includes a minimal graph assertion store, but that should not be confused with graph-based retrieval or an ontology reasoner. A comparative GraphRAG

study would require an additional retrieval path and evidence that the path was actually used. Similarly, the option of local inference does not establish that every data flow is local; embeddings, authentication, storage, and chat configuration must be considered independently.

## 1.6. Dissertation structure

The achieved result is a committed integration of governance, retained evidence, and access-aware retrieval with an advisory knowledge console, accompanied by repeatable engineering checks of a later local working state. The checks passed in their declared environments; controlled model comparison and human outcomes have not been measured. Chapter 2 develops the conceptual foundation and critical literature synthesis. Chapter 3 explains the requirements, implementation, and the boundary between committed and subsequent local changes. Chapter 4 presents executed checks, requirement-level evidence, and the protocol for missing effectiveness evaluation. Chapter 5 answers the implementation-led questions and defines the work required before stronger claims are warranted. The appendices provide reader-facing scenarios, evaluation instruments, and evidence limitations.

CHAPTER 2

# State of the Art

This chapter examines the literature needed to interpret Sentinel KMS as an implemented cybersecurity knowledge-management artefact. The synthesis proceeds from organisational knowledge sharing to cybersecurity information models, retrieval and generation, and their evaluation and security constraints. The literature explains the rationale and trade-offs of the current design; it does not supply a feature list that the implementation is assumed to satisfy or establish that a generative interface is preferable to ordinary search.

## 2.1. Review approach and evidence boundaries

### 2.1.1. Inputs and scope

The starting material comprises two connected project research notes, a research plan, and a claim matrix. The detailed note describes KMS-related publications; the recommendation supplies the wider accepted-reference list and summarises technical systems and counterevidence. The earlier research plan restricted discovery to IEEE Xplore and Mendeley. During preparation of this dissertation, DOI metadata and selected abstracts were checked through bibliographic services and accessible publisher records. This subsequent checking is not represented as a repeat of the original discovery search.

The supplied recommendation reports that an earlier process received 105 candidates and retained 100 after excluding five records with insufficient abstract evidence. Those are handoff-reported screening figures, not screening decisions independently reproduced in this dissertation. Parsing the actual accepted list verifies 100 distinct URLs: 90 DOI links and ten Mendeley catalogue links. The earlier figure of 95 DOI-bearing candidates relates to the candidate pool, not to the accepted list. The catalogue retained with this draft preserves that distinction.

The discussion is a critical, literature-informed synthesis rather than a completed systematic literature review. The available materials do not provide enough independently checked search histories, full-text eligibility decisions, or study-quality assessments to justify a new PRISMA diagram or a claim of exhaustive coverage. Screening figures describe the supplied research materials, not an independently replicated review.

### 2.1.2. Levels of support

Three evidence levels are distinguished. A verified bibliographic record establishes the identity of a publication. A retrieved abstract can support a bounded account of what the authors say they studied and reported. Inspection of full-text methods can support a more detailed methodological description, but does not constitute replication or an exhaustive quality appraisal. The RAGAs and ARES methods were inspected directly for this draft; much of the remaining discussion relies on abstracts or the explicitly labelled KMS handoff.[6, 18]

Unresolved identifiers remain in the machine-readable reference catalogue rather than receiving invented titles or author lists. They are not used as evidence for substantive claims in the dissertation. The complete supplied bibliography is retained for subsequent research, while the printed reference list includes the sources actually cited. Appendix B describes the evidence records and how to distinguish retrieval from interpretation.

## 2.2. Knowledge management and cybersecurity knowledge sharing

### 2.2.1. A KMS is more than a question-answering interface

Ipe's conceptual paper places knowledge sharing at the level of interactions between people inside organisations.[9] Yeboah's review distinguishes knowledge-sharing enablers, processes, and outcomes, and argues that sharing processes should relate to organisational objectives and resources.[26] These works do not evaluate a cybersecurity chatbot. Their relevance here is conceptual: a technically accurate answer is not equivalent to knowledge being contributed, trusted, reused, or integrated into a decision.

For Sentinel KMS, this distinction motivates a workflow that retains the document collection and contribution process independently of the assistant. A user should be able to locate and inspect a source without accepting a generated interpretation. A contributor should be able to supply or revise knowledge through a governed path rather than having conversational output silently become authoritative knowledge. These are design choices derived from the KMS perspective, not effects demonstrated by those organisational studies.

Osman et al. review ontology-based knowledge-management tools using comparison dimensions that include knowledge sources, extraction, input, retrieval, sharing technology, and ontology methodology.[15] Their framework is useful because it places retrieval within a larger information lifecycle. Applied to this project, it suggests evaluating the costs and failure modes of contribution and maintenance alongside the quality of answers. For example, a highly accurate assistant would still be operationally problematic if approved corrections did not reach its retrieval index or if analysts could not determine who was responsible for a source.

### 2.2.2. Trust and organisational context in CTI

The supplied KMS handoff characterises the survey by Wagner et al. as addressing technical and non-technical CTI-sharing problems, including trust, relevance, and organisational constraints; its full text was not independently appraised for this draft.[23] More recent abstract evidence from Abraham et al. presents a perspective on CTI sharing informed by interviews with cybersecurity professionals and emphasises the interaction of human insight, incompatible platforms, and different meanings of trust.[1]

A practical implication is that provenance should not be reduced to a URL. A source can be identifiable but inappropriate for the user's environment, out of date, or available only to a restricted group. Conversely, information from an internal incident review may be highly relevant but unsuitable for an external model provider. This thesis therefore treats authority, access class, and temporal scope as separate attributes rather than collapsing them into a single relevance score.

These constraints also affect evaluation. Contribution frequency, reuse of approved information, evidence inspection, and acceptance of unsupported advice concern different aspects of a KMS. A positive satisfaction rating cannot by itself establish that an analyst selected a correct mitigation. The participant protocol in Chapter 4 consequently separates perceived usefulness from decision quality and potentially harmful reliance.

## 2.3. Ontologies and cybersecurity knowledge graphs

### 2.3.1. Semantic organisation and interoperability

Takahashi and Kadobayashi describe a reference ontology intended to organise cybersecurity operational information and coordinate industry information schemata. Their abstract situates this work in information exchange across organisational boundaries and describes a prototype knowledge base.[22] Mavroeidis and Bromander analyse CTI taxonomies, sharing standards, and ontologies, arguing for broader, layered coverage of threat intelligence.[13] These contributions motivate a semantic model that can accommodate different kinds of security knowledge without treating every record as an interchangeable text passage.

Sikos reviews graph-based data models and knowledge-organisation systems used to represent cybersecurity knowledge, including their roles in aggregation, visualisation, machine learning, and reasoning.[20] Bratsas et al. similarly place common structured representation and interoperability at the centre of their review of semantic-web tools for CTI.[4] Neither review, as used here, establishes that adding a graph to Sentinel KMS will improve answer accuracy. They explain what a structured layer may represent and which capabilities would require evaluation.

For this dissertation, an ontology denotes the conceptual vocabulary and constraints used to describe knowledge, whereas a graph denotes particular stored entities and relations. Merely storing entity and edge tables is not evidence of a validated domain

ontology, standards conformance, or automated logical reasoning. The inspected Sentinel graph is a minimal evidence-linked assertion store; it is a limited implementation step towards structured knowledge, not the complete realisation of the semantic capabilities surveyed in the literature.

### 2.3.2. Dynamic knowledge and source revision

Pastuszuk et al. motivate a Dynamic CyberSecurity Ontology by observing that real IT systems change and require current information about their components and risks. Their abstract outlines monitoring and aggregation from dynamic sources.[16] Kougioumtzidou et al. describe an AI-assisted framework with distinct taxonomy construction, ontology construction, and updating phases.[10] Together these sources support treating updates as part of the information model rather than as occasional administrative maintenance.

The design consequence for Sentinel KMS is to separate an article's identity from a particular revision and the evidence passage drawn from that revision. A later publication should not silently change what an earlier answer cited. At the same time, retaining a historical reference does not mean that a user remains authorised to open it. Historical identity and current permission solve different problems and must coexist. This is an architectural inference developed in the project, not a lifecycle implementation copied from either cited paper.

Temporal correctness also has at least two meanings. A response may accurately describe what a source said at a past date, yet be unsuitable as current advice. Evaluation must specify which meaning is requested. A question about a current mitigation requires current applicable evidence; a retrospective investigation may require an older revision with a clearly stated date. An index update alone cannot establish either interpretation without explicit case expectations.

### 2.3.3. Graph quality, inferred edges, and chunking

Li et al. explicitly identify noise and conflict introduced during graph construction as potential causes of misleading inference and decision-making, and propose a quality-assessment approach.[11] Wang et al. investigate graph completion using ensemble learning and adversarial training.[24] Their abstracts support the existence of graph-quality and incompleteness problems, not the proposition that a confidence score or completion model resolves them for every cybersecurity task.

A sourced assertion and an inferred relation should therefore be distinguished. The former records that an author claims source support; the latter records an inference and its method. Neither label is an automatic endorsement of truth. An inferred relationship that later appears in an answer must not acquire the apparent authority of an explicitly documented relationship solely because both are represented as graph edges. Sentinel's minimal graph records this distinction, while the evaluation of the semantic validity of assertions remains separate.

SC-LKM associates hierarchical semantic chunking with cybersecurity graph construction and reports comparisons involving entity coverage, topology density, and semantic consistency.[25] The abstract motivates investigating whether chunk boundaries preserve useful relationships. Sentinel's current text chunker instead splits content into overlapping character windows, trims each window, and drops empty results. That simple mechanism is not hierarchical semantic chunking. A semantic-chunking comparison would require a new implementation and controlled measurement before the literature's proposed benefits could be attributed to this project.

## 2.4. RAG and language models for cybersecurity work

### 2.4.1. Relevant systems and their different tasks

MoRSE describes a cybersecurity question-answering system with parallel retrieval over different information representations. Its indexed abstract reports improved relevance and correctness in its own comparison.[21] This is evidence of a relevant research direction, but the reported advantage is not an expected effect size for Sentinel KMS: the present draft has not independently appraised the complete benchmark, source overlap, or model configuration.

GenDFIR addresses incident timeline analysis through preprocessing, incident-event structuring, retrieval, and an LLM. Its abstract explicitly locates the evaluation in synthetic cyber incident events in a controlled environment.[12] This is an important distinction because successful reconstruction of a prepared scenario is not equivalent to a successful live investigation. Sentinel may use analogous illustrative scenarios to test information flow, but must label them as such.

RAGIntel proposes hybrid retrieval, reranking, and compression for attack investigation using structured threat intelligence. Its abstract reports performance comparable to standalone LLMs, despite motivating retrieval as a way to improve the quality of investigation support.[2] That qualification should be retained rather than replacing the authors' comparison with a generic claim that RAG is superior.

### 2.4.2. Counterevidence and baseline selection

The PyPI malicious-source-code study reports that RAG performed poorly relative to few-shot learning in the authors' evaluation.[8] ForensicLLM reports that a locally deployable, quantised, fine-tuned model outperformed its tested base and RAG models for digital-forensics questions.[19] These results do not prove that few-shot learning or fine-tuning is universally better. They establish that a literature review cannot infer the best architecture merely from the presence of positive RAG studies.

A further comparison of three models for incident detection and analysis reports uneven benefits from retrieved knowledge. Its abstract describes evaluation against a GPT-4o-generated reference using cosine similarity and BERTScore.[14] This introduces

Table 2.1: Selected technical studies and their bounded implications. Reported outcomes below are abstract-level claims, not replications.

| Study | Task and reported direction | Implication for Sentinel KMS |
|---|---|---|
| MoRSE[21] | Cybersecurity QA; reports gains in its comparison. | Contextualise combined retrieval; do not equate Sentinel's lexical/vector fusion with this system or import its advantage. |
| GenDFIR[12] | Timeline analysis on synthetic incident events. | Keep controlled demonstration separate from field effectiveness. |
| PyPI detection study[8] | Malicious source-code detection; few-shot learning outperforms the tested RAG approach. | Include alternatives and do not assume additional context helps. |
| ForensicLLM[19] | Local fine-tuned digital-forensics model; reports advantages over its base and RAG comparators. | Motivate deployment comparison without implying that Sentinel implements fine-tuning. |
| RAGIntel[2] | Attack investigation; reports performance comparable to standalone LLMs. | Measure the value of governance and evidence access separately from accuracy gains. |

a specific interpretive limit: agreement with a model-produced reference is not the same construct as independent expert assessment of a security decision. The numerical scores from that study are consequently not used to set Sentinel's acceptance thresholds.

The planned baseline comparison therefore starts with search-only, no-RAG generation, and RAG on the same cases. A local model condition is valuable where data handling and computational constraints make it relevant. Sentinel already implements optional reranking of authorised candidates, making an enabled/disabled comparison possible in principle; no benefit is established here. Fine-tuning and graph retrieval are not current experimental conditions and would require additional implementation. A stronger model with a larger prompt budget should not be described as evidence for a superior retriever if multiple variables change at once.

### 2.4.3. Local deployment and the unit of comparison

ForensicLLM demonstrates that local deployment is a concrete design candidate in digital forensics.[19] It does not demonstrate that local inference automatically resolves privacy, maintenance, or reliability concerns. In Sentinel KMS, chat generation, embedding generation, authentication, and document storage are independent components. The exposure of a query or document depends on the configured route through those components, not solely on whether the chat model runs locally.

The appropriate experimental unit is thus the complete declared configuration. Model identity, quantisation, prompt, retrieval settings, corpus revision, hardware, and provider route should accompany every result. This requirement also applies to cost: an unreported provider charge is missing information, not zero cost, and locally incurred computation

still consumes resources even when no per-request bill exists.

## 2.5. Evaluating retrieval, generation, and human outcomes

### 2.5.1. Component-level evaluation

The RAGAs paper distinguishes faithfulness, answer relevance, and context relevance. Its method decomposes an answer into statements and asks whether those statements are supported by the supplied context; the resulting score is a model-assisted estimate rather than direct proof of correctness.[6] A relevant answer can still be unsupported, and an answer faithful to an incorrect source can still be factually wrong. This distinction is particularly important when the source corpus itself is under governance review.

ARES uses trained lightweight judges for context relevance, answer faithfulness, and answer relevance, together with human annotations for prediction-powered inference.[18] Its design illustrates the role of human reference judgements in calibrating scalable evaluation. It does not provide a basis for changing pending Sentinel expert-review fields to complete simply because an automated diagnostic was computed.

Retrieval metrics address a different question: whether relevant material is returned and ranked where it can be used. An article-level retrieval hit may still omit the passage necessary to support a particular claim. Consequently, this thesis distinguishes article relevance, passage support, generated claim correctness, and citation entailment. Sentinel's deterministic answer evaluator uses phrase and provenance checks explicitly labelled as heuristics; it is not an implementation of the RAGAs or ARES judge methods. Those papers justify the conceptual separation of evaluation dimensions, not a claim that calibrated semantic scoring is already available. Chapter 4 gives the operational definitions and the treatment of empty or unanswerable cases.

### 2.5.2. Abstention, conflicting evidence, and negative cases

The Retrieval-Augmented Generation Benchmark (RGB) studies noise robustness, negative rejection, information integration, and counterfactual robustness. Its abstract reports difficulties in rejecting unsupported requests, integrating information, and dealing with false information.[5] These findings motivate a test set containing more than questions for which a convenient answer is already present.

An abstention can be correct when accessible evidence is insufficient, but it can also conceal a retrieval failure when suitable evidence exists. The evaluation must therefore score the reason for abstention against the case definition. Similarly, a response that states a conflict may be more appropriate than a single confident recommendation when the corpus contains incompatible sources. Neither behaviour should be judged solely by lexical overlap with one preferred answer.

For access-controlled cases, the reference evidence is the evidence available to the requesting identity. A document that is relevant but restricted should not be counted as

a missed authorised hit. At the same time, the system should not reveal the contents or even unnecessary existence information about restricted material while explaining why it cannot answer. These are project-specific evaluation requirements derived from the interaction of relevance and authorisation.

### 2.5.3. Human outcomes and the limits of automation

Bernard and Balog's review of fairness, accountability, transparency, and ethics in information retrieval distinguishes different evaluation approaches, including audits and user studies for accountability and transparency.[3] Abraham et al.'s CTI-sharing perspective likewise indicates that sharing outcomes cannot be reduced to data-format interoperability.[1] These sources support looking beyond automatic answer scores, but do not supply an already validated questionnaire for this exact prototype.

The proposed analyst study measures decision quality and elapsed time together with confidence, evidence inspection, and acceptance of unsupported advice. Its purpose is not to ask only whether users like the interface. An assistant could make answers faster to produce while increasing the severity of mistakes or making uncertain advice appear more trustworthy. Only a controlled study with representative tasks and independently assessed outcomes can resolve that possibility for Sentinel KMS.

## 2.6. Security and governance of retrieved knowledge

Greshake et al. demonstrate indirect prompt injection through attacker-influenced information that an LLM-integrated application may retrieve. Their abstract identifies the blurring of instructions and data as a central source of the attack surface.[7] For a cybersecurity KMS, a retrieved report can therefore contain both relevant security facts and text attempting to redirect the assistant. A trusted-looking document location is not sufficient evidence that every instruction-like sentence should be obeyed.

The design response is to keep evidence distinguishable from instructions and place authorisation outside the model. This is a defence-oriented architectural choice, not a claim that delimiters or prompt wording prevent all injection. Evaluation must examine the configured system's behaviour on hostile passages and preserve failures. Future graph retrieval would require analogous tests for misleading entities and relations; adding structured data would not remove the need for an adversarial corpus model.

Knowledge governance also concerns ordinary mistakes and legitimate changes. A previously approved article can expire, a user can lose workspace membership, or an author can issue a correction. The system must reconcile persistent evidence identifiers with live read permission. Audit records can assist investigation of such events, but their value depends on what is actually persisted and on failure behaviour. Chapter 3 therefore distinguishes audit call sites from a guarantee of durable audit delivery.

## 2.7. Synthesis and research direction

The reviewed material supports a bounded research direction with three commitments. First, the durable object is governed knowledge: sources, revisions, passages, assertions, and the conditions under which they can be used. Second, retrieval and generation are independently replaceable mechanisms whose benefits depend on a controlled comparison. Third, evidence integrity and human consequences must be evaluated separately from output fluency.

The central literature conflict remains unresolved by design. Relevant studies report positive, neutral, and adverse comparisons for retrieval depending on the task and configuration.[2, 8, 21] The thesis does not average these results across incomparable settings or select only favourable findings. It uses them to justify an empirical question and a baseline design. Similarly, graph representations motivate semantic organisation but require their own quality controls, while scalable answer metrics require calibration before they can support high-impact security decisions.[11, 18]

These conclusions frame the implementation analysis in Chapter 3. Knowledge-sharing research explains why the library and publication workflow remain distinct from chat. Work on dynamic knowledge motivates retained revisions and explicit lifecycle state, while graph-quality research supports distinguishing sourced and inferred assertions without claiming graph-based retrieval. RAG research contextualises the implemented lexical/vector fusion and optional reranking; counterevidence makes their benefit a question for measurement. Evaluation research explains why provenance checks and phrase matches cannot replace expert judgments. The resulting design rationale is therefore tied to existing mechanisms, with semantic chunking, graph retrieval, fine-tuning, and demonstrated analyst benefit kept outside the current contribution.

CHAPTER 3

# Sentinel KMS: Design and Implementation

## 3.1. Committed implementation and working-tree interface

The inspected `main` branch ends at commit `8d259eb` (13 September 2026). That commit already contains the Spring/React knowledge console, Supabase bearer authentication, PostgreSQL with pgvector, hybrid retrieval, governed publication, retained-source ingestion, immutable evidence references, an exact-passage viewer, and a minimal graph assertion store. The checked-out tree also has uncommitted backend and interface changes: in particular, revised response prompts and fallback guidance, a response-plan intake, playbook browsing, and redesigned navigation. These are separate from the committed contribution. This chapter attributes the system mechanisms below to committed `main`; it marks subsequent working-tree behavior explicitly rather than treating a local screenshot or test as evidence that those additions were committed, deployed, or used by analysts.

Table 3.1: Source-inspected provenance. Governance, durable ingestion, immutable evidence, and graph storage already exist in committed `main`; the right column concerns additional uncommitted work.

| Area | Committed `main` (`8d259eb`) | Additional working-tree changes |
|---|---|---|
| Retrieval and answers | Policy-filtered hybrid candidates, indicator lookup, optional reranking, revision-bound citation tuples, and rechecked passage viewing. | Revised incident-response prompts, source-based fallback on generation failure, and response-plan intake. |
| Ingestion and feeds | Retained upload/URL sources, leased jobs, fenced indexing, and a queued refresh route that publishes changed feed records as revisions. | Redesigned navigation and presentation in the captured interface; no new ingestion guarantee is inferred. |
| Knowledge governance | Workspace classification, approval, publication, expiry, revision and read policies. | Playbook browsing and other local knowledge-view changes; the committed governance boundary remains the reference. |
| Graph and evaluation | Evidence-linked assertion store (not GraphRAG), synthetic evaluation cases, and review gates. | No additional graph-retrieval feature or human-effectiveness result is evidenced by the local changes. |

This distinction follows inspection of the committed assistant orchestration, ingestion and feed services, governance and evidence schema, and graph store, compared with the local diff. Appendix A records the source provenance. An implemented call path establishes neither a successful production rollout nor an effectiveness result; the screenshot walkthrough below exercises the current working-tree frontend against fixtures, not the committed backend in a live deployment.

In committed `main`, an article identifier displayed to a reader is a compatibility presentation of a server-resolved revision and evidence span, not independent proof that the answer's claim follows from that span. The server's identity and authorization checks address provenance and disclosure; semantic support still requires the separate, pending expert adjudication described in Chapter 4.

Consider a *design scenario* in which an editor proposes a revised mitigation playbook. An administrator other than the proposer approves it; reindexing makes the approved revision retrievable under its access policy. An authorised analyst asks about the mitigation, receives an advisory answer with a citation to a particular revision and passage, and opens that passage for review. If the publication later expires or access is revoked, the same historical reference remains recorded but no longer discloses the passage. This example connects the intended control boundaries. It is not a recorded end-to-end execution or an analyst observation; the browser demonstration later in this chapter uses unrelated, fixed mock responses rather than publishing its uploaded fixture into the cited corpus.

### 3.2. Scope and implementation requirements

Committed `main` implements a cybersecurity knowledge console in which document management, retrieval, and conversational assistance share an authorization boundary. Its central design problem is not simply finding text resembling a question. A useful answer must draw on material that remains readable by the requesting analyst, identify the revision that supplied the evidence, and avoid treating an unpublished proposal as operational guidance. The account is grounded in committed definitions and call paths, not an idealized architecture; local interface refinements are identified separately. Appendix A provides detailed implementation provenance.

The implementation supports requirements at three distinct levels. Analysts need searchable documents, recoverable ingestion, and answers with inspectable evidence. Knowledge custodians need publication controls, restricted access, and revision history. Operators need configurable infrastructure and records of consequential operations. Table 3.2 distinguishes four functional requirements (F1–F4) from three cross-cutting, non-functional requirements (N1–N3). Chapter 4 returns to the same identifiers with separate evidence and outstanding work rather than treating mechanism presence as operational effectiveness.

The application is organized as a Spring backend, a React frontend, and PostgreSQL

Table 3.2: Functional (F) and non-functional (N) requirements, implementation mechanisms, and their boundaries.

| Requirement | Mechanism | Important qualification |
|---|---|---|
| F1 Governed knowledge | Membership, classification, approval, and lifecycle checks | Authentication alone does not grant workspace access. |
| F2 Recoverable ingestion | Retained sources, leased jobs, and fenced replacement | Durable processing is not an exactly-once provider-call guarantee. |
| F3 Relevant retrieval | Vector and lexical candidates fused by rank | Ranking configuration is not a measured quality result. |
| F4 Traceable answers | Immutable revision and evidence identities with passage offsets | A valid citation does not prove that a claim follows from it. |
| N1 Access and disclosure safety | Policy checks at retrieval and passage view; withheld responses for unavailable evidence | Isolated denial tests do not certify adversarial or production security. |
| N2 Reviewable operations | Redacted, append-only audit records | Audit persistence is best-effort. |
| N3 Deployment flexibility | Separate database, identity, and AI configuration | Local AI does not imply an entirely local deployment. |

storage with pgvector. Backend packages separate knowledge management, ingestion, assistant orchestration, and indexing. This separation matters because representations have different authority: an article is a mutable reading projection, a revision preserves content, a vector row is a retrieval derivative, and a citation identifies evidence. Neither a browser-supplied identifier nor vector metadata independently establishes permission.

### 3.3. Runtime architecture and deployment boundaries

The backend exposes authenticated HTTP operations and uses JDBC alongside repository-based persistence. The committed database migrations define the schema; the base application configuration disables Hibernate schema changes and automatic vector-store initialization. It also specifies cosine distance, an HNSW index type, and 1024-dimensional embeddings. Those settings describe compatibility requirements, not measured index performance. A replacement embedding model must preserve the configured dimensionality or be accompanied by a schema and reindexing change.

Identity and data hosting must be distinguished from model hosting. The local-AI configuration selects Ollama for both chat and embeddings. It does not create an identity service or relocate the database. Conversely, hosted Supabase supplies database and authentication services only when deployment variables point to that project. The browser requires its own Supabase URL and public client configuration; the backend requires

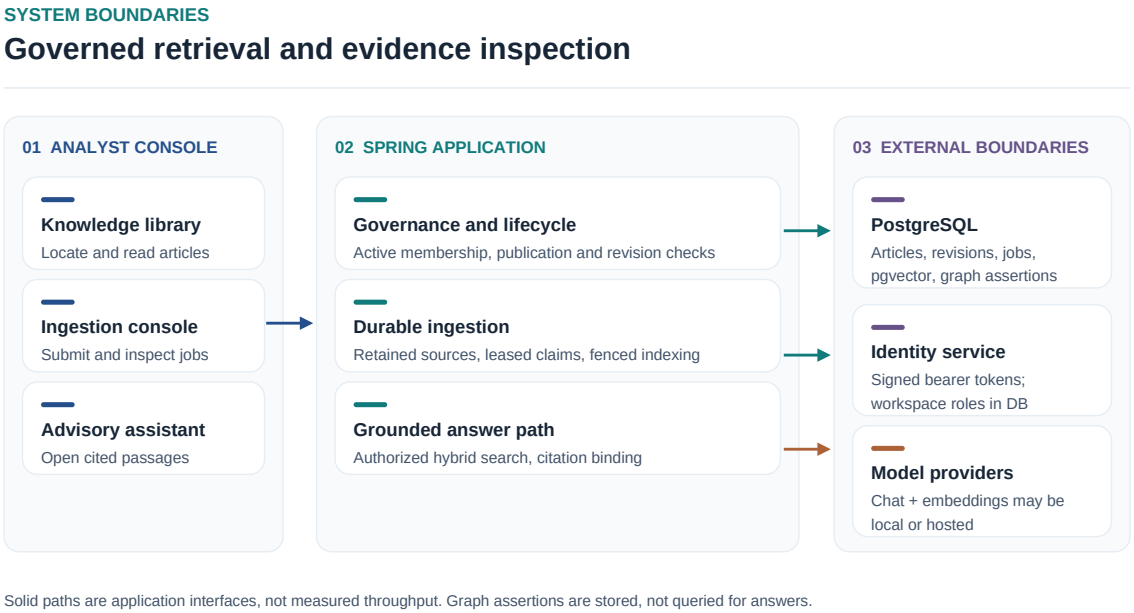



Figure 3.1: Conceptual boundaries in committed `main`. Arrows represent service interfaces, not a measured deployment or a claim that graph assertions enter answer generation.

database connectivity and a reachable signing-key endpoint.

The distinction is visible in the container deployment definition. Its database is a local pgvector container, whereas authentication uses an external Supabase endpoint or a separately running local service. Ollama belongs to the optional local-AI profile. The default backend service selects the local model profile but points to the Ollama service name; starting only the core services therefore does not by itself provide every model dependency. Furthermore, the base application configuration selects Mistral when its profile variable is absent. There is consequently no single universal default topology: environment substitution and the selected launch mechanism must be recorded for reproducibility.

The security configuration implements stateless bearer-token authentication. It validates signatures and token timestamps, applies an audience check when configured, and checks the issuer only when its setting is nonblank. Most requests require authentication, with explicit exceptions for health, API documentation, and preflight requests. These are implemented controls, not evidence that a particular hosted installation uses an appropriate issuer, database role, or network perimeter.

## 3.4. Governance and knowledge lifecycle

### 3.4.1. Identity, classification, and authorization

The governance policy distinguishes public, internal, confidential, and restricted articles. These classifications correspond respectively to public, workspace-member, workspace-

editor, and workspace-admin read policies. Public still requires an authenticated context. An inconsistent classification-policy pair is denied rather than interpreted as a permissive override. Missing governance, incomplete policy fields, unapproved content, and inactive lifecycle states also fail closed.

The knowledge-access service resolves workspace authority from active database memberships. This avoids accepting a role merely because a client includes it in a request. Membership lookup failure supplies no workspace roles. Subject-aware checks are reused by article access and assistant evidence resolution; retrieval SQL independently applies the corresponding database predicates. This duplication provides enforcement at different boundaries but also creates a maintenance obligation: changes to policy semantics must remain consistent across application logic and SQL.

Uploads are a separate authorization case. Their durable source identifier, source-revision identifier, and source key must resolve to an indexed, nondeleted, noncancelled job. Owner-private and workspace-governed sources are not interchangeable. An upload is not promoted into an approved article merely because extraction succeeds, and its metadata cannot manufacture an article citation.

### 3.4.2. Publication as a controlled state transition

The knowledge-lifecycle service separates immutable content from mutable workflow state. Editors can create draft revisions and submit them for review. Approval is an administrative operation and rejects self-approval. Creating or rejecting a proposal does not overwrite the currently readable publication. This permits an existing playbook to remain available while its successor is assessed.

Lifecycle operations lock governance rows and compare an expected state version. The resulting conflict behavior protects against applying an action to a publication state that changed after the caller last inspected it. Reviews and lifecycle events preserve the transition history, while workflow projections describe current approval and lifecycle status. The creation-time status stored in an immutable revision must not be mistaken for its current publication status.

Supersession approves and publishes a pending-review successor through the controlled publication path; it is not a pointer switch to an already approved replacement. Like ordinary approval, this operation requires an administrator and rejects self-approval. Rollback follows a different administrative path: it creates a new monotonically numbered revision from a previously published revision, copies historic evidence into fresh identities, and publishes it without a new pending-review cycle. It does not redirect the current pointer to an old row. Expiry retains history while removing present read eligibility. Retrieval and evidence queries also compare expiry timestamps with the database clock, so historical retention does not imply continuing access.

Publication and vector availability are deliberately decoupled. A durable reindex outbox connects the lifecycle transaction to the knowledge reindex worker. The worker

passes the revision and governance version to fenced replacement, cancels stale work, and retains retryable failures. Old vectors cannot justify an answer simply because new vectors are not ready. The availability consequence is an interval in which a published revision may lack searchable embeddings; the integrity benefit is that stale evidence is not silently substituted.

Trusted feed publication is an explicit exception to manual review, not an implicit property of every imported document. The trusted-article publisher initializes known server-side producers as public, approved knowledge; initial article persistence and revision/evidence registration share a transaction. On a source-owned record, a changed snapshot can create a new immutable revision without overwriting manually governed content. This is a configured trust decision about producers, not proof that every imported assertion is correct.

### 3.5. Immutable evidence and provenance

The governance schema introduces revision snapshots and evidence snapshots. Revisions preserve article identity, revision number, classification, content hash, authorship, and lineage. Evidence rows retain source labels, optional source URI and version, content, and a hash. Mutation-rejection triggers protect revision and evidence history against ordinary update and delete operations.

An article evidence reference combines article ID, revision ID, revision number, evidence-snapshot ID, and passage boundaries. URI and source version add provenance metadata but do not replace that identity. The retrieval joins verify that stored chunk content equals the specified substring of the immutable snapshot. Thus a plausible title, current article ID, or forged metadata tuple is insufficient to authorize arbitrary text as evidence.

Immutability has a limited meaning here. It is enforced within the database and application model, not by an external notarization service or tamper-proof hardware. A hash identifies retained content; it does not establish the truth of its claims, its completeness, or the authenticity of the upstream publisher. Some trusted evidence is materialized from normalized article fields rather than an archival copy of the original feed response. The implementation should therefore be described as preserving its own evidence representations, not universally preserving every original source byte.

Historical provenance also differs from historical readability. The exact-evidence viewer resolves a reference against the current approved revision and returns only the requested passage. Invalid, stale, unauthorized, revoked, and unavailable references converge on an unavailable result. A citation can remain durably recorded even when this viewer will no longer disclose its content. This favors present authorization over unconditional replay of previously visible material.

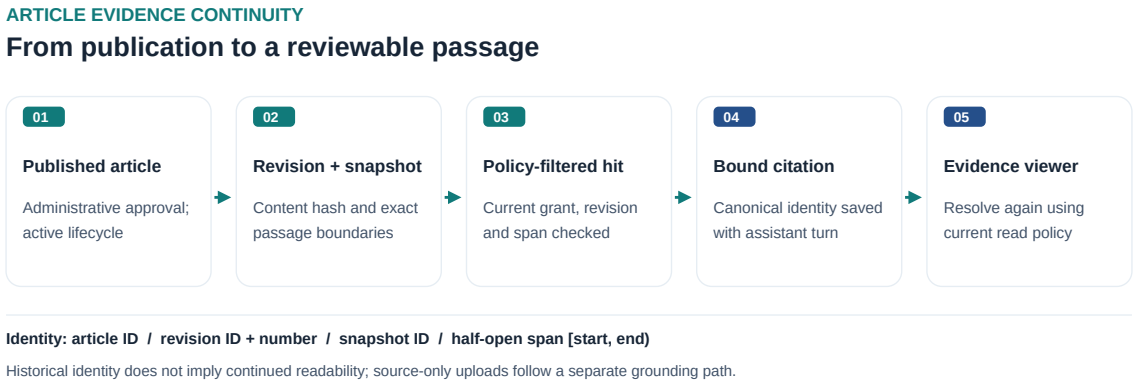



Figure 3.2: Evidence continuity for a governed article, from publication through retrieval and citation to a subsequent access-checked passage view. Upload-only grounding is a separate path; the diagram does not assert that a citation proves answer faithfulness.

## 3.6. Durable ingestion and source integration

### 3.6.1. Admission, retention, and processing

The ingestion admission service creates durable jobs rather than performing the full pipeline during admission. Uploads undergo extension and size checks, then receive server-generated source and revision identities. Optional request idempotency is owner-scoped and associates a key with a request fingerprint and job. A repeated request can recover its existing job; omitting the key preserves separate-request semantics. This mechanism is distinct from worker retry and should not be described as global document deduplication.

URL admission validates the address before storing the request. The URL guard rejects non-HTTP schemes, embedded credentials, and blocked address classes. Network restrictions reduce the risk of turning ingestion into access to internal services, but they are not a complete evaluation of every deployment's outbound-network security.

The ingestion worker retains a successful URL response before extraction so retries can reuse it instead of silently substituting a later response. It checks the durable source binding, extracts text, sanitizes it, chunks it, requests embeddings, and completes indexing. Missing bytes and empty extraction cause explicit failure; no synthetic document is substituted. Apache Tika performs extraction with a character cap, while the chunker uses overlapping character windows rather than sentence segmentation or token-aware semantic partitioning. Configured window size and overlap are 1200 and 150 characters.

Sanitization appears both before upload indexing and during prompt construction. It targets recognizable injection patterns and frames documents as untrusted data. These measures should be understood as defensive processing, not a proof that prompt injection has been eliminated. Extraction and sanitization can also alter the text representation, which reinforces the need to distinguish retained bytes from derived retrieval text.

### 3.6.2. Concurrency and connector boundaries

The durable ingestion queue claims jobs using PostgreSQL row locks with skip-locked selection. A claim includes a random token, lease, and index generation. Recovery advances the generation; progress and heartbeat operations require a matching live claim. Final index replacement checks ownership of the claim again before changing stored chunks. These controls address a specific race: a slow worker must not overwrite newer work after recovery, retry, or deletion.

This is a fenced durable workflow, not an assertion of exactly-once execution. Embedding calls can occur before a final claim becomes stale, and a retry may repeat external work. The intended invariant concerns which replacement may commit, not whether a provider was contacted only once.

Feed refresh follows a separate, committed article-publication route. The active connector endpoint checks database-backed mutation authority and queues NVD, CISA KEV, or SANS ISC refresh; an optional schedule uses the same persisted per-source queue. The feed worker stores a lease, source checkpoint, counters, and bounded run history. It processes source-scoped identifiers, leaves unchanged content without a new revision, and publishes changed source-owned records as fresh immutable revisions and evidence. This publication queues the knowledge reindex worker, so a completed fetch does not mean the revised article is already searchable. Existing manually governed or edited records are not silently overwritten, and watchlist matching runs for new or changed records. Bounded changes per run and a partial state disclose a possible backlog rather than proving complete feed coverage. An older NVD-only helper still contains skip-existing logic, but it is not the refresh route invoked by the connector endpoint.

## 3.7. Access-aware hybrid retrieval and answer generation

### 3.7.1. Candidate selection and ranking

The embedding index service implements vector and lexical retrieval directly against PostgreSQL. The vector branch embeds the question and orders candidates by cosine distance. The lexical branch uses the simple text-search configuration, web-search query parsing, and text-search ranking. Category and minimum-severity filters are applied in both branches. Authorization joins and predicates are evaluated before each branch's candidate limit, rather than removing unauthorized rows only after an unrestricted top-$k$ selection.

Article candidates must bind to current approved, active, unexpired governance, compatible workflow state, and an exact evidence span. Upload candidates instead require qualifying durable job state and source access. The minimum-score parameter constrains vector similarity, not lexical rank or the final fused score. Scope predicates use article category and severity fields; they do not implement equivalent upload-specific category and severity metadata. Vector-branch failure leaves the lexical branch available, but the lexi-

cal fallback repeats the authorization conditions; failure does not activate an unrestricted query.

The hybrid ranker merges the branches by reciprocal-rank fusion. For a candidate $d$ appearing in one or both ranked lists, the ordering score is

$$F(d) = \sum_{\ell \in \{\text{vector,lexical}\}: d \in \ell} \frac{1}{60 + \text{rank}_\ell(d)}. \tag{3.1}$$

This avoids directly equating cosine similarity with lexical rank. An implementation nuance is that the merge returns the retained branch row after sorting by fusion score; its exposed score is not replaced with $F(d)$. Neither value is a calibrated probability of correctness. No improvement over either individual branch is established merely by implementing fusion.

### 3.7.2. Grounding before generation

The assistant orchestration service combines hybrid retrieval with subject-aware indicator lookups. It then intersects article candidates with readable, existing articles and resolves exact evidence identities. Source-only uploads pass their separate database-backed gate. Optional reranking receives this authorized material, not unrestricted retrieval results. The indicator branch receives the literal question and user subject but not the hybrid branch's category or minimum-severity filters. These scope controls therefore must not be described as universal constraints on all assistant grounding.

Indicator grounding also differs from a verbatim retrieved passage. The indicator lookup locates the indicator value in an approved evidence snapshot, while the assistant constructs a summary from article and indicator fields. Its evidence span identifies the indicator occurrence, not every title, severity, or CVSS assertion in that constructed summary. This is another reason why structural citation resolution does not establish claim-level support.

The service first attempts literal-question grounding, then permits conversational rewriting and optional reranking if evidence exists. If grounding remains empty, it returns a local abstention instead of invoking answer generation. An important qualification concerns provider exposure: hybrid retrieval can already call the embedding model for the question. The implemented abstention gate therefore prevents subsequent rewriting, reranking, and answer-generation calls in the empty-grounding path; it is not a universal promise of zero provider calls.

The prompt builder constructs bounded retrieval context, includes recent conversation material, and labels sources as untrusted. The nonstreaming chat path requests structured JSON answer blocks; the SSE path uses a separate natural-language system prompt with the same context builder. Both instruct the model to copy server-provided evidence tokens rather than invent identifiers. The configured context-character budget applies to the assembled source section, not the entire prompt, and can truncate the final source. As a

result, stored grounding provenance must not be interpreted as proof that every referenced character was supplied to, or influenced, generation. The more prescriptive incident-response prompts and source-based generation-failure fallback belong to the uncommitted working tree, not to this committed path.

### 3.8. Citations, presentation, and analyst inspection

Citations are a server-resolved contract rather than arbitrary model annotations. The orchestration binds retrieved article spans to a citation resolver, sanitizes generated tokens, and persists canonical references with the assistant message. It distinguishes provenance supplied as grounding from citations actually declared in the response. Authorized uploads may support an answer without possessing the article evidence identity required for the article citation viewer.

Authorization is checked again before final output. The streaming implementation buffers provider deltas, sanitizes the completed sequence, rechecks evidence, and persists the response before exposing its safe events. Although the endpoint uses server-sent events, this path should not be represented as immediate provider-token streaming. Its buffering trades responsiveness for a final disclosure check. Failed final authorization produces a local abstention rather than releasing the generated text.

The committed React interface connects these mechanisms to analyst tasks. Its query layer covers the knowledge base, conversations, and periodically refreshed ingestion jobs. The assistant view integrates an evidence dialog into answer turns and binds the selected evidence to the current user. The dialog validates the exact identity, checks that the response matches it, and displays an unavailable state on mismatch or failure. It cancels obsolete requests, manages focus, and makes the background inert while open. Source links are restricted to suitable HTTP(S) URLs; source metadata does not authorize content disclosure. The response-plan form, playbook-browsing panel, and revised navigation visible in the local build are uncommitted interface changes, not prerequisites for the committed evidence-view contract.

These mechanisms support inspection but do not establish usability or faithfulness. A reader must still assess whether the displayed passage supports the answer's claim. The frontend's statement that answers are grounded in the corpus expresses intended behavior, not an experimental guarantee that every sentence is supported.

Figures 3.3–3.6 show four selected states of the current, locally modified frontend in a scripted browser walkthrough. The browser intercepted GoTrue authentication and application API requests with deterministic fixtures; it asserted the visible article, upload response, cited answer, exact-passage response, and request counts. The images are not evidence of a live database, an executed backend ingestion or model call, or a human analyst study. The identifier `CVE-2025-1111`, index counts, answer, and passage are artificial. The uploaded fixture does not supply the cited answer; a separate fixed response does. Crops highlight the relevant controls and text.

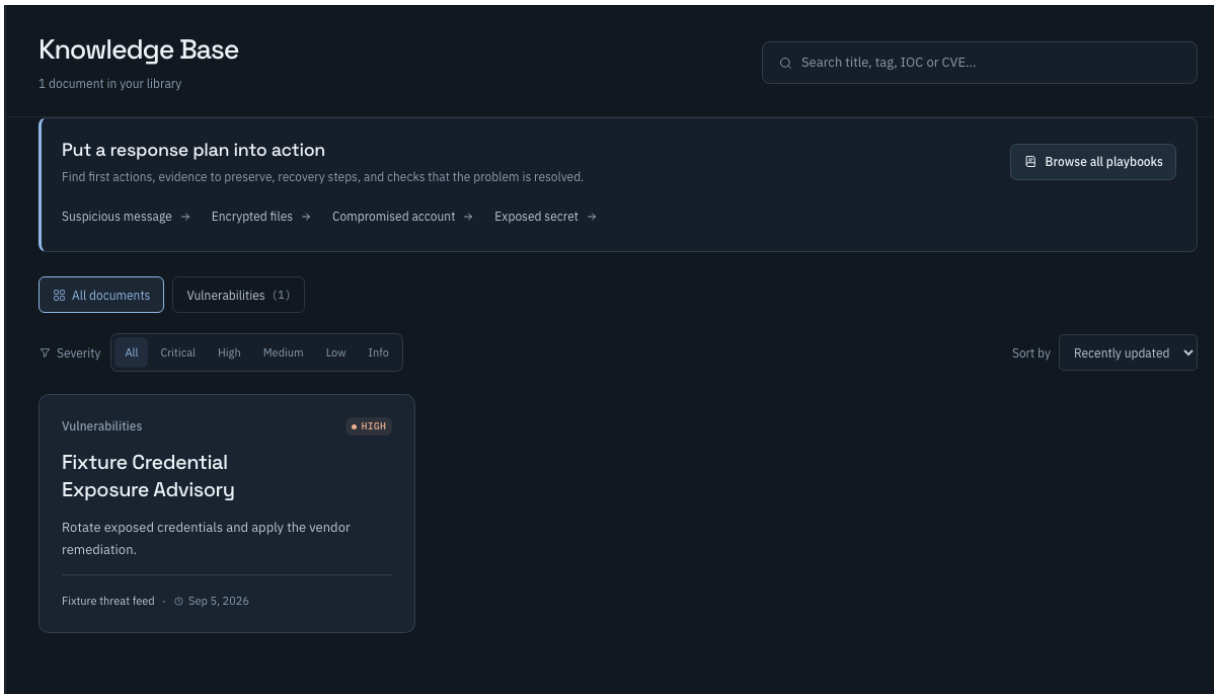



Figure 3.3: Knowledge-base collection in the modified frontend: a playbook-browsing band above one fixture advisory. The band is an uncommitted interface addition; the displayed document and count are mocked, not a measured corpus.

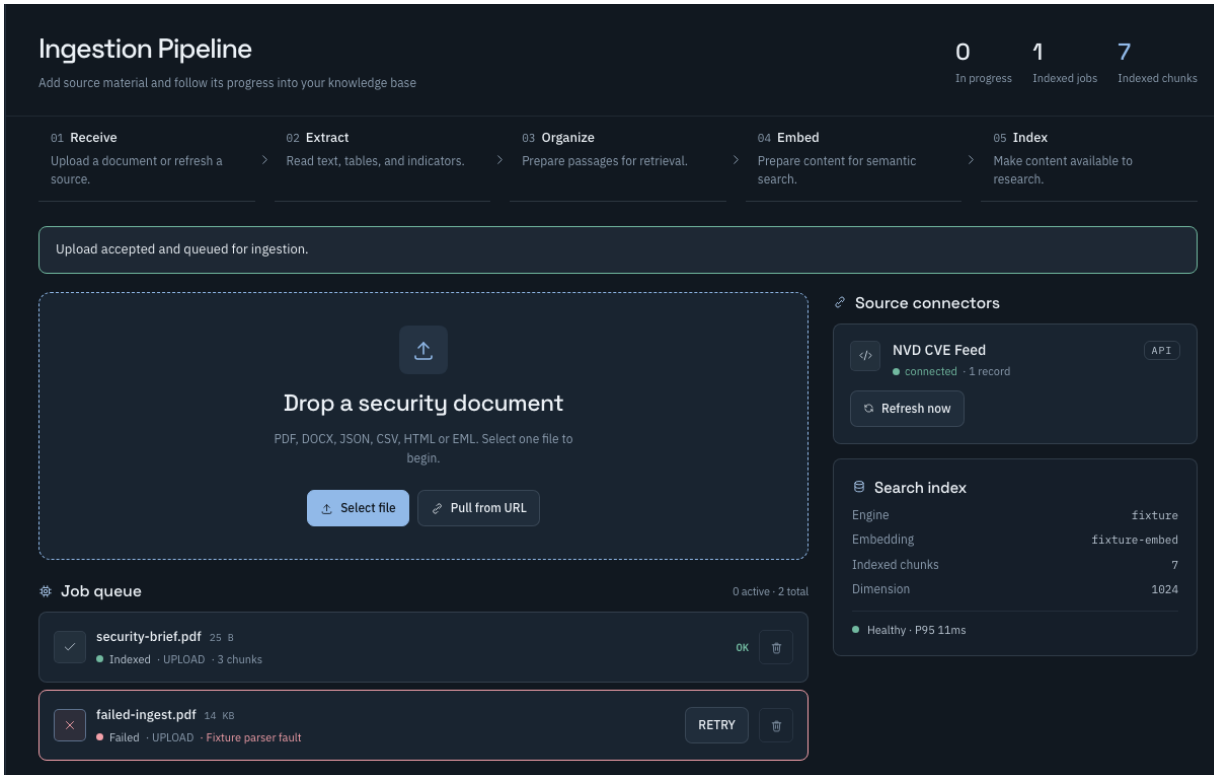



Figure 3.4: Ingestion view after the browser submitted a fixture upload. The accepted message, indexed and failed jobs, connector status, and index counters come from intercepted API responses; the fixture immediately reports "indexed" without a backend worker having processed the file.

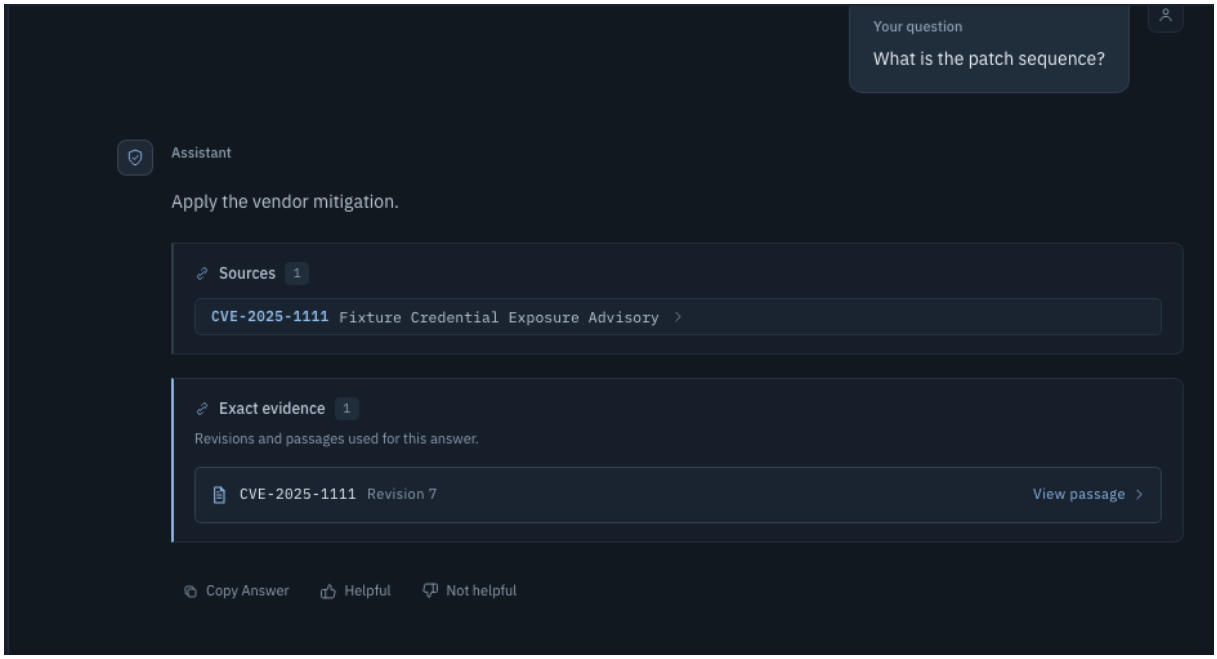



Figure 3.5: Assistant reply with a fixture article citation and a distinct revision 7 passage action. The answer text and SSE frames are intercepted responses, not a measured language-model run or proof that the cited passage entails the advice.

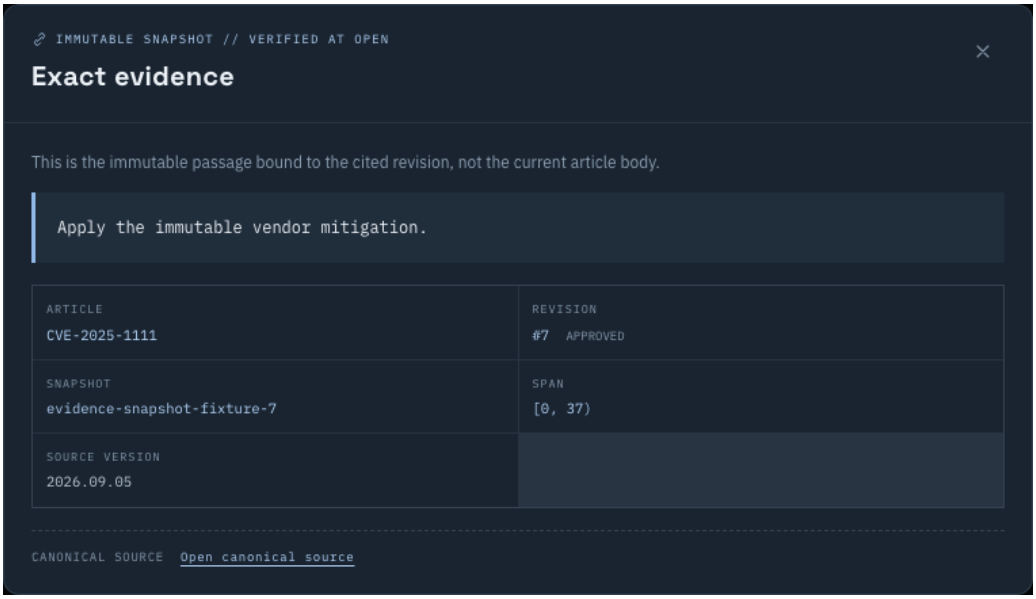



Figure 3.6: Focused exact-evidence dialog with the fixture passage, revision, snapshot identity, span, and canonical-source action. The response is mocked; the browser contract does not exercise the committed server's authorization or establish source authenticity.

### 3.9. Graph assertions and operational audit

The graph is a *minimal assertion store*, not graph traversal or GraphRAG. Its API exposes entity and edge creation and individual reads. The graph service binds assertions to readable article evidence, requires editor authority for creation, and checks compatible workspace and classification at edge endpoints. Sourced and inferred assertions remain distinct; inferred records require method and version metadata. The schema preserves evidence hashes, observation time, and recording time.

These fields document assertions rather than independently validate them. The inspected assistant retrieval path does not consume graph entities or edges, and the graph controller contains no neighborhood expansion or path-query endpoint. Claims about multi-hop reasoning, graph-enhanced recall, or automatic inference would exceed this implementation.

Operational auditing uses the knowledge-audit service and the audit-hardening schema. Metadata is sanitized, and database triggers reject audit mutation, including truncation. Successful events inside a transaction are deferred until commit; terminal events use an independent transaction. However, audit insertion failures are caught and logged in reduced form. This protects the original operation from audit-related disclosure or exception masking, but it also permits missing events. The implementation is neither a guaranteed-complete compliance ledger nor an externally anchored cryptographic chain.

### 3.10. Limits of the implementation account

This chapter establishes what committed `main` implements in inspected definitions and call paths, with the additional frontend and prompt work marked as a local, uncommitted snapshot. It does not establish that every database migration is installed in a running hosted project, that infrastructure secrets and roles are correctly provisioned, or that the system withstands all adversarial inputs. The accompanying implementation audit distinguishes source inspection from the separate offline validator execution.

Test code supplies useful evidence of intended invariants. For example, the governed hybrid-search regression test captures SQL and checks policy and evidence-binding clauses using mocked dependencies. Such tests are not retrieval-effectiveness experiments, and merely reading them is not executing them. No pass count, response-quality improvement, analyst time saving, or production reliability result is inferred here. Chapter 4 must distinguish executed checks from planned or source-inspected validation. The contribution described in this chapter is the concrete integration of governance, durable ingestion, access-aware retrieval, and inspectable provenance, with the stated limits preserved.

CHAPTER 4

# Evaluation Design and Current Evidence

## 4.1. Purpose and evidential boundary

The evaluation must distinguish a functioning implementation from a useful and safe decision-support system. Software tests can establish that specified inputs produce specified outputs; they cannot establish that generated security advice is correct, that a citation entails a claim, or that analysts make better decisions. This chapter therefore separates executable regression evidence, inspection of evaluation mechanisms, and a prospective empirical protocol. The project's evaluation protocol informs the design, but its existence is not evidence that a study occurred.

The present contribution is an evaluation design backed by engineering verification of the inspected, modified working tree on 29 September 2026. The current checks exercised a synthetic-report quality gate, Java tests including isolated container-backed cases, frontend unit tests, scripted Chromium journeys, and a production frontend build. An earlier audit also exercised the same report validator, but its result is not added to the current total. Neither execution is a live retrieval benchmark, a provider comparison, an expert annotation exercise, or an analyst study. Accordingly, no measured model ranking, improvement in task completion, participant count, or significance claim is reported. Missing empirical results remain missing; engineering thresholds and synthetic inputs are not substitutes. Appendix A explains the evidence classes and the external verification records retain execution details.

The implementation-led questions in Chapter 1 determine the evidence boundary. RQ1 is addressed by tracing revision, passage, policy, and lifecycle identities across the source definitions described in Chapter 3. RQ2 is addressed by distinguishing the assistant's runtime grounding and citation path from the narrower hybrid-search evaluator and from capabilities not connected to generation. RQ3 requires examining stale-evidence, authorization, and hostile-content controls without promoting their presence into an operational-security guarantee. This chapter directly addresses RQ4 by separating executed validator checks from unexecuted mechanisms and the empirical work still required.

That subsequent empirical work has three dimensions: whether retrieval delivers the authorized evidence needed for a security question; whether generation preserves its meaning, scope, and temporal qualification; and whether assistant-supported work improves analyst decisions without increasing unsafe reliance. These are proposed effectiveness

questions, not replacements for the implementation-led research questions or findings of this thesis. They require different units of observation: ranked evidence, generated claims, and analyst–case decisions. A successful result at one level must not be transferred automatically to another. The component distinction is informed by RAGAs and ARES; the authorization and proposed analyst-study requirements are specific to this project.[6, 18]

## 4.2. Available artifacts and executed checks

### 4.2.1. Datasets and evaluation mechanisms

The retrieval evaluation dataset contains ten cases, including severity- and category-scoped questions. Expected relevance is expressed as article identifiers. The separate answer evaluation dataset contains nine synthetic cases: two answerable cases and one each for unanswerable, ambiguous, conflicting, stale, multi-document, adversarial, and authorization scenarios. Temporal disclosure is represented within an answerable case. Both the answer dataset's overall review record and all nine case records remain pending, with no reviewer identity or review timestamp supplied.

These are regression fixtures, not a representative incident sample. Their expected phrases and evidence identifiers encode developer assertions. In particular, the stale fixture requires abstention even though its question asks about a superseded snapshot; whether that is the appropriate response to a historical question requires rubric review. The fixture must not silently become a normative rule that historical evidence can never be discussed.

The evaluation mechanisms separate retrieval scoring, deterministic answer checks, saved-output comparison, optional provider execution, and external annotation import. The answer checker examines phrases, citation provenance, temporal markers, and forbidden identifiers. Its explicit heuristic labels are essential: matching a required phrase is not semantic correctness, and omission of a forbidden citation identifier does not rule out uncited disclosure. Immutable revision and span fields support traceability but do not by themselves prove entailment.

### 4.2.2. What the offline run establishes

An earlier, isolated Python report-validator audit completed twelve tests successfully. On 29 September 2026, the same twelve-test report-gate suite also passed on the current modified working tree. The tests use temporary synthetic reports in subprocesses; neither application startup nor a live retrieval request is involved. The earlier pass is not an additional twelve current cases, and the current pass is not a component of the Java, frontend, or browser totals. Test elapsed time is suite duration, not retrieval or model latency.

The fixtures exercise acceptance of structurally healthy retrieval and JUnit reports, and rejection of low recall, empty reports, non-finite metrics, inconsistent totals, duplicate

identifiers, false pass declarations, skipped tests, missing required suites, and concealed testcase failures. Even the test named for an executed integration report uses a synthetic XML document. Its success establishes report-validator behavior, not that a database integration test ran. The numeric retrieval scores inside the test helper are likewise fabricated test inputs, never observations of the assistant.

The gate requires every declared retrieval case to pass and checks aggregate recall, MRR, and nDCG against defaults of 0.8, 0.6, and 0.6. It does not independently reconstruct those aggregate values from ranked evidence or bind a report to a frozen corpus. Its role is bounded validation of declared results. The report gate, including its current rerun, must not be conflated with the separate application suites below or with provider-backed retrieval assessment.

### 4.2.3. Executed software verification and scripted walkthrough

Table 4.1 reports the 29 September 2026 engineering checks against the modified working tree, not the committed baseline. A green test outcome means the asserted software behaviour held for the tested inputs and environment; it is not a sampled human outcome or a measure of generated-answer accuracy. The Java suite includes isolated container-backed PostgreSQL checks alongside mocks, but no production database or hosted model was involved. At least the illustrative screenshot journey in the Chromium suite intercepts authentication and application responses; it does not run the complete ingestion-to-generation path.

Table 4.1: Observed engineering verification on the modified working tree, 29 September 2026; test counts are not participants or evaluated security questions. The report gate is a distinct synthetic-check layer, not part of the application-suite totals.

| Layer | Observed result | What this result cannot establish |
|---|---|---|
| Report quality gate | 12 passed on current rerun | Fabricated retrieval and test reports exercise validator logic, not live retrieval quality. |
| Java tests | 408 in 88 reports; 0 failures, errors or skips | Isolated containers and mocks do not certify a hosted deployment or expert answer quality. |
| Frontend unit tests | 91 passed in 13 files | Component behaviour in a test DOM does not prove backend authorization. |
| Chromium browser tests | 17 passed | The mocked capture demonstrates an interface contract, not real provider, authentication or database behaviour. |
| Frontend production build | Succeeded; 133 modules transformed | Compilation alone does not verify live integration or answer quality. |

For the illustrative figures, a scripted analyst persona follows a fixed sequence: sign in through intercepted test authentication, upload a fixture document, ask a prepared mitigation question, and open the article-revision passage. The browser asserts the upload

status, cited answer, exact evidence text, canonical-source link, and request counts before recording the figures. The uploaded file does not cause the cited answer: intercepted application responses independently supply a fixed article and answer. Other contract cases exercise unavailable evidence, job retry, watchlist creation, expired sessions, and permission denial. These are simulated interactions, neither human-subject interviews nor evidence that an analyst understood or trusted the interface appropriately. The displayed indexed state in the capture is returned by a mock response, not by an ingestion worker.

The current Java test reports were counted by parsing 88 test-result documents: 408 tests, with no failures, errors or skips. Earlier test counts and a previous unsupported-runtime frontend failure describe a different verification snapshot and are not substituted for these current results. The report-gate rerun passed twelve of twelve fixture tests; the frontend test runner reported 91 passed tests across 13 files; the Chromium runner reported 17 passed tests; and the production build transformed 133 modules without error. The exact commands, environments, timestamps, and report artifacts belong in the external verification record. None of these counts is a live RAG benchmark.

### 4.3. Prospective corpus and comparison design

Before empirical execution, freeze the source documents, revision statuses, extraction and chunking configuration, embedding identifier, index snapshot, retrieval configuration, prompts, generation parameters, and access-policy snapshot. Preserve hashes and a manifest connecting case identifiers to authorized passages. A router alias alone is insufficient model identification when the selected underlying model can change; retain the actual identifier when available and disclose unresolved routing variability.

Construct development and held-out sets before tuning. Split related incidents and document families together so that paraphrases or adjacent revisions do not leak between sets. Include routine questions as well as difficult and negative cases, stratified by answerability, evidence multiplicity, recency, and authorization boundary. Domain reviewers must judge relevance against the evaluator's permitted corpus, not a global corpus containing inaccessible material. Otherwise an access-control success could be misclassified as retrieval failure.

Compare three conditions on identical case identifiers. Search-only exposes ranked authorized passages without generated synthesis. No-RAG uses the same generation model and response budget without retrieved organizational context. RAG adds the frozen retrieval context to that model. Search-only should be assessed through evidence finding and downstream task decisions; lack of generated prose is not an answer-quality failure. Local and hosted models form an additional comparison only for cases whose data policy permits both. Analyze the common eligible subset separately rather than presenting policy-driven exclusions as model superiority.

Record context and output budgets, temperature, stopping rules, prompt versions, and query-rewrite behavior. Evaluate optional reranking through a paired on/off abla-

tion that holds the candidate pool constant. A separate end-to-end comparison may include its additional latency and calls. The existing graph represents assertions, not an active graph-retrieval condition; neither graph-enhanced recall nor a graph ablation is currently available. Any future graph condition must first demonstrate which graph-derived evidence enters the response path.

Store raw outputs before scoring in restricted storage, with public reports carrying artifact bindings rather than sensitive answer text. The saved-output runner requires complete baseline case coverage, while provider execution requires explicit opt-in. These mechanisms facilitate comparisons but do not certify that supplied baseline outputs were generated under controlled conditions. In particular, the HTTP comparison adapter sends the question, baseline name, and fixture evidence to a caller-supplied endpoint for every baseline, including no-RAG. It does not itself invoke the application's assistant pipeline or enforce context removal for no-RAG. A controlled comparison therefore requires an independently verified endpoint implementation and request traces; a baseline label alone is insufficient. Retain execution manifests, failures, and configuration evidence outside the scoring report as necessary.

## 4.4. Metrics and operational definitions

### 4.4.1. Retrieval quality

Let $R_q$ be the independently judged relevant article set for question $q$, and let $L_q = (d_1, \ldots, d_m)$ be the ordered, deduplicated retrieved articles. For cutoff $k > 0$, define $h_i = \mathbf{1}[d_i \in R_q]$ and $H_q(k) = \sum_{i=1}^{\min(k,m)} h_i$. The proposed precision and implemented recall definitions are

$$P_q@k = \frac{H_q(k)}{k}, \qquad R_q@k = \frac{H_q(k)}{|R_q|}. \tag{4.1}$$

Precision here treats unfilled slots as non-relevant; report the number returned alongside it. Recall is undefined for an empty relevant set in the scientific analysis, so those cases receive a separate negative-retrieval analysis. The implementation returns zero for empty relevance sets; that convention must not be mistaken for an expert judgment of unanswerability.

Define reciprocal rank and its case-wise mean as

$$\mathrm{RR}_q = \begin{cases} 1/r_q, & r_q = \min\{i : h_i = 1\} \text{ exists}, \\ 0, & \text{otherwise}, \end{cases} \qquad \mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{RR}_q. \tag{4.2}$$

The implementation scans the returned ranked list for reciprocal rank. With binary

relevance, normalized discounted cumulative gain is

$$\mathrm{nDCG}_q@k = \frac{\sum_{i=1}^{\min(k,m)} \frac{h_i}{\log_2(i+1)}}{\sum_{i=1}^{\min(k,|R_q|)} \frac{1}{\log_2(i+1)}}. \tag{4.3}$$

Again, the scientific analysis separates cases with a zero denominator. Macro-averages give each eligible question equal weight; stratified results expose concentrated failures that an overall mean could hide.

The retrieval evaluator invokes authenticated hybrid search with scope filters, deduplicates article identifiers, and marks a case as passing whenever recall exceeds zero. Thus its pass flag means at least one expected article appeared, not that every required source was retrieved. Chunk-level top-$k$ selection precedes article deduplication, so the final list may contain fewer than $k$ articles. Source-only upload hits are omitted from article-level scoring. The endpoint shares the hybrid-search component with the assistant, not its complete grounding pipeline: it does not add indicator-index matches, run conversational rewriting or reranking, perform the assistant's subsequent evidence-resolution steps, or assess generated answers. Precision, passage-level relevance, and independent-source diversity are proposed extensions, not existing reported endpoint metrics.

Execution permission is separate from document readability. The evaluation controller checks dataset viewing and retrieval execution against access-service grants and active workspace membership. Execution needs both dataset-view and execution grants in the same workspace, rather than merely an authenticated bearer token or an editor role. The evaluator then passes the requester's identity to hybrid search and enforces a configured maximum case count. These checks do not adjust the fixture's expected article identifiers to the evaluator's permissions. Before measuring retrieval, reviewers must therefore confirm that each expected set is appropriate for that identity; an operational grant does not make otherwise protected documents readable.

### 4.4.2. Answer quality and safety

Reviewers should segment an answer into material claims $C_q$ and identify claim–citation links $E_q$. Let $s(c)$ indicate support by authorized, temporally appropriate evidence, and $e(c,p)$ indicate that cited passage $p$ entails claim $c$. Define

$$\mathrm{Support}_q = \frac{\sum_{c\in C_q} s(c)}{|C_q|}, \qquad \mathrm{CitationAccuracy}_q = \frac{\sum_{(c,p)\in E_q} e(c,p)}{|E_q|}. \tag{4.4}$$

Also report citation coverage: the proportion of evidence-requiring claims linked to adequate supporting passages. Empty claim or citation sets are not automatically perfect; classify abstentions separately and mark inapplicable ratios explicitly. Correctness addi-

tionally requires completeness, appropriate scope, and absence of materially misleading omissions. A supported but incomplete answer can still fail the task.

Let $a_q$ indicate required abstention and $b_q$ observed abstention. Report both

$$\text{AppropriateAbstention} = \frac{\sum_q a_q b_q}{\sum_q a_q}, \qquad \text{OverAbstention} = \frac{\sum_q (1 - a_q) b_q}{\sum_q (1 - a_q)}. \tag{4.5}$$

Undefined denominators remain unreported with counts explained. Temporal correctness requires the response to distinguish current approval, historical state, uncertainty, and supersession, not merely repeat a date. Unsafe-action rate is the fraction of evaluable responses containing reviewer-confirmed harmful recommendations; report severe failures separately from minor errors. None of these expert-scored quantities is measured by the Python run.

For efficiency, collect elapsed wall-clock time, timeouts, token usage, and provider-reported cost for each attempted case. Distinguish time to first output from completed response latency. Report missing cost as unknown, not zero, and separate local inference resource consumption from provider billing. Availability and failure counts accompany quality summaries so that selective successful responses cannot inflate apparent performance.

### 4.5. Expert review and adversarial validation

Two qualified reviewers should independently validate the dataset and rubric, then review each baseline–case candidate without seeing the competing scores. Record reviewer identifiers, qualifications, dates, evidence inspected, rationales, and seven outcome labels corresponding to the annotation schema. Resolve disagreement or uncertainty through a separate adjudicator. Report pre-adjudication agreement and its denominator; adjudicated labels do not demonstrate that independent agreement was originally high.

The external annotation importer and scientific-completeness gate provide a procedural boundary. Reviews bind to dataset and output hashes, candidate digests, baseline identifiers, case identifiers, and immutable evidence tuples. The gate requires independent accepted reviews and appropriate adjudication; missing external review records cannot be replaced by fixture review metadata. However, structural checks cannot authenticate reviewer qualifications or prove actual independence. Those responsibilities remain organizational. A complete review containing failing quality labels is still a complete review, not evidence of satisfactory performance. All external reviews remain pending for the supplied dataset.

Adversarial evaluation should combine held-out malicious-source variants with benign controls. Test indirect instructions inside documents, forged authority statements, misleading citations, contradictory revisions, irrelevant retrieved passages, ambiguous scope, and attempts to obtain inaccessible evidence across users or workspaces. Use synthetic canaries rather than real secrets. Record whether protected content reached retrieval,

prompts, outputs, logs, or citation views; a clean final answer alone does not establish that access boundaries held.

The existing denied-reference fixture checks a forbidden identifier, while its adversarial fixture checks specific required and forbidden phrases. Both can miss paraphrased violations and can penalize harmless quotation. Their deterministic results therefore remain diagnostic. Real authorization validation requires an isolated deployment with distinct identities, database-authoritative permissions, request traces, and controlled provider-call observation. Safety testing must not execute operational recommendations against production assets. Preserve failed cases and stop a study on predefined severe disclosure or harmful-action criteria.

### 4.6. Statistical and ethical protocol

Pre-register a primary outcome and a practically meaningful effect before estimating sample size. A defensible pilot or external estimate should inform prospective power analysis; the repository supplies no justified participant count. For case-level model comparisons, retain pairing across identical cases and repetitions. Bootstrap uncertainty at the independent case or incident-family level, not at the level of correlated claims. Report effect estimates and confidence intervals alongside raw counts, with multiplicity control for confirmatory secondary comparisons and exploratory analyses clearly marked.

The analyst study requires institutional or organizational approval, informed consent, and a documented recruitment procedure with experience strata. Use a counterbalanced within-participant comparison of ordinary search and assistant-supported work, assigning different matched case sets to reduce recall effects. Record allocation seeds, task order, elapsed time, decision correctness, confidence before and after, opened evidence, overrides, and acceptance of unsupported advice. Blind outcome reviewers to condition where feasible. Account for participant and case clustering in the analysis rather than treating repeated decisions as independent observations.

Define timeout, withdrawal, infrastructure failure, and exclusion rules prospectively. Retain the assigned-case denominator and show missingness by condition, supplemented by sensitivity analyses rather than imputing invented answers. Establish retention periods and access controls for raw outputs and study records. Keep consent records and identifiable participant information outside the repository. Confidential material must not reach hosted providers without explicit approval for its data class. Tasks remain advisory and sandboxed, with severe errors recorded independently of average usability or speed.

### 4.7. Requirement validation and limitations

Table 4.2 returns to the implementation requirements in Chapter 3, Table 3.2. The F1–F4 identifiers denote functional capabilities; N1–N3 denote cross-cutting access safety,

auditability, and deployment flexibility. N1 makes the access/disclosure boundary explicit rather than treating governance alone as proof of runtime safety. For each requirement the table links an inspected mechanism and selected test evidence to an honest current status and pending acceptance step. The scenario and review instruments in Appendix C turn those gaps into testable questions. The source-level and test-level results concern RQ1–RQ3; their separation from effectiveness evidence addresses RQ4.

The principal limitations are small synthetic fixtures, pending expert judgments, and no measured model outputs or analyst observations. Article-level hits may miss relevant passages, while phrase checks can reward superficial compliance. Provider drift, corpus changes, policy differences, and learning effects would constrain later comparisons. The opt-in workflow and CI gates do not prove a deployment passed them. The executed suites establish bounded engineering behaviour; effectiveness, comparative model quality, and operational safety remain open.

Evaluation integrity is an additional research obligation, not evidence that any requirement has been empirically accepted: the current twelve-test report-gate rerun checks synthetic report consistency, while the application suites check selected implementation paths. Authentic expert reviews, a controlled live comparison, and approved observations of analyst decisions have not been supplied. The graph assertion store is not a graph-retrieval mechanism, and neither reranking nor graph benefit is inferred from these tests.

Table 4.2: Requirement ID to implementation/test evidence and current status, linked to Chapter 3, Table 3.2. Passing tests establish selected assertions only; empirical acceptance remains pending.

| Requirement ID | Implementation and test evidence | Current status and next acceptance |
|---|---|---|
| F1 Governed knowledge | Membership, classification, approval and expiry gates; policy and lifecycle tests passed in the Java suite. | Engineering-checked, not operationally validated. Pending: multi-identity publication and revocation trials. |
| F2 Recoverable ingestion | Retained sources, leased jobs and fenced replacement; ingest/recovery tests include isolated PostgreSQL checks. | Engineering-checked. The screenshot upload is intercepted; failure-injection and sustained recovery trials remain pending. |
| F3 Relevant retrieval | Access-filtered vector–lexical search and article-level scoring; hybrid-search component tests passed. | Component-checked, effectiveness unmeasured. Ten relevance cases are developer fixtures; freeze independently judged rankings and compare baselines. |
| F4 Traceable answers | Revision-bound passages, citation binding and rechecked evidence viewer; citation/viewer tests passed and the browser capture was mocked. | Engineering-checked, entailment unvalidated. Independent claim–passage and temporal review remains pending. |
| N1 Access and disclosure safety | Policy-filtered retrieval, final evidence recheck and unavailable-state handling; authorization tests passed in isolation, while browser denial is mocked. | Partially checked, not an operational security guarantee. Pending: live-state identity boundaries, injection variants and disclosure traces. |
| N2 Reviewable operations | Redacted audit with protected stored rows and best-effort writes; unit and isolated database tests passed. | Mechanism checked, completeness unvalidated. Pending: event-loss monitoring and operational audit reconciliation. |
| N3 Deployment flexibility | Separate identity, storage and model configuration; configuration tests and frontend build passed. | Build/configuration checked, hosting unvalidated. Pending: deployment matrix, provider/data-flow audit and policy approval. |

CHAPTER 5

# Conclusion

## 5.1. Conclusions supported by the present work

This dissertation has developed an account of Sentinel KMS as a governed cybersecurity knowledge-management artefact. The design problem is not exhausted by connecting a language model to a vector index. An answer depends on which sources are available, which revisions are current, which passages support the response, and which user is permitted to inspect them. The inspected implementation gives those concerns explicit representation in document governance, retrieval, answer persistence, and evidence viewing.

The principal research question is answered at the level of implementation and bounded verification, not operational effectiveness. The committed system integrates explicit source and revision identities, database-backed access decisions, governed retrieval, and server-resolved evidence references. Later local work adds curated response playbooks, an incident-oriented interface, and evidence-dependent fallback behaviour, but those changes are not part of the inspected commit. Source inspection identifies the mechanisms; Java, frontend, browser, and report-validation checks establish only the behaviour exercised in their declared environments. None certifies a deployed system.

The subordinate questions have correspondingly bounded answers. RQ1 is addressed by the distinction between retained source or revision identity, derived retrieval text, and later evidence resolution under current permissions. RQ2 is addressed by hybrid retrieval and indicator lookups, authorised grounding, optional reranking, local abstention on missing grounding, and canonical article citations; the graph assertion store is not a retrieval branch. RQ3 is addressed by lifecycle and access checks, fenced indexing, repeated evidence resolution, and defensive prompt processing, with deployment security and injection resistance still requiring adversarial execution. RQ4 is addressed by the separation of deterministic evaluation mechanisms and executed report checks from the prospective provider, expert, and analyst evaluations in Chapter 4.

These answers also expose implementation trade-offs. An immutable reference can become unavailable under current access rules. A published revision can temporarily lack new embeddings. Authorised uploaded material can support grounding without having the article identity required by the article citation viewer. A passage supplied as context need not support every generated sentence. These are substantive characteristics of the implementation, not merely documentation details.

Whether the design improves analyst work remains unanswered. No controlled model comparison or participant study was conducted for this account. The passing engineering checks and scripted browser journeys do not measure retrieval quality, answer correctness, or analyst performance. The dissertation therefore does not conclude that Sentinel improves incident response, reduces analyst workload, or provides safe autonomous security operations.

## 5.2. Main contributions

The first contribution is the implementation account of knowledge governance integrated with durable ingestion and an advisory retrieval interface. Classification, workspace authority, approval, lifecycle state, and exact evidence identities are treated as part of knowledge access rather than as optional annotations appended to generated text. Source inspection identifies enforcement paths in the current code, while the evidence audit preserves their qualifications. The contribution lies in this integration and its analysis, not in proposing a new retrieval algorithm.

The second contribution is the explicit distinction between knowledge representations. Document content, immutable revisions, evidence snapshots, vector rows, graph assertions, and answer citations serve different purposes. In particular, the graph stores evidence-linked assertions but is not a graph-retrieval engine. This distinction prevents the presence of graph tables from becoming an unsupported claim of GraphRAG capability or multi-hop reasoning.

The third contribution is a layered evaluation procedure. It separates article retrieval, passage support, generated claims, citation links, abstention, temporal scope, access control, and analyst decisions. This is consistent with the component-level evaluation concerns raised by RAGAs and ARES, while adding the project-specific obligations created by governance and security-sensitive information.[6, 18] The procedure also preserves pending expert reviews and incomplete evidence instead of treating syntactically valid reports as completed scientific validation.

Supporting these contributions, the thesis package provides a traceable writing basis: the source-note exports, bibliographic catalogue, retrieved evidence, implementation audit, and offline test record accompany the LaTeX document. This supports subsequent correction and empirical extension. It does not replace the author's responsibility to inspect the cited work and verify the final interpretation.

## 5.3. Limitations

### 5.3.1. Literature and argument

Much of the literature is available here at abstract or handoff level. Such evidence supports descriptions of research directions and reported findings but leaves important questions about study design, sampling, leakage, baselines, and statistical uncertainty unresolved.

The synthesis is not an exhaustive systematic review. Unresolved bibliographic records are preserved transparently and do not support the substantive argument.

The literature also addresses different tasks: general knowledge sharing, CTI representation, question answering, malicious-code detection, and digital forensics. Their results cannot be pooled into a single predicted improvement for Sentinel. Positive and negative comparisons motivate controlled experiments rather than a presumption in favour of retrieval.[8, 19, 21]

### 5.3.2. Implementation and deployment

The description covers a committed implementation and distinguishes later local changes, neither of which is a certified deployment. The configured issuer, database permissions, outbound network rules, provider routing, and installed schema must be verified in each deployment. Local chat inference does not by itself establish local processing of embeddings, authentication, or storage.

The current graph has no retrieval or traversal integration. The ingestion pipeline's durable coordination does not guarantee that external work occurs exactly once. Evidence hashes and immutability controls preserve internal representations rather than guaranteeing source authenticity. Audit persistence is best-effort, and retained events therefore do not establish complete accountability. None of these limits invalidates the prototype, but all constrain what it can be claimed to demonstrate.

### 5.3.3. Evaluation and generalisation

The fixtures are small and synthetic, with pending expert review. Their phrases and identifiers encode intended behaviour rather than independently established ground truth. The stale-evidence case also exposes a substantive rubric question: whether a historical query should require abstention or a qualified historical answer. That decision must be resolved before the fixture is used to score correctness.

There are no measured provider-backed comparisons or analyst outcomes in this draft. Even after such measurements are collected, generalisation would be limited by the sampled users, documents, organisations, task definitions, and configurations. Severe errors must remain visible alongside average quality and speed; a favourable mean cannot establish that rare harmful advice is acceptable.

## 5.4. Future work and completion priorities

The first priority is evidence preparation rather than additional architectural complexity. The author should obtain and appraise the full texts most central to the argument, resolve remaining bibliographic records where possible, and review the corpus and case definitions with qualified cybersecurity reviewers. Review independence, qualifications, and adjudication must be documented outside the mere structure of an annotation file.

The next priority is an isolated, reproducible evaluation deployment. It should contain a frozen, authorised corpus and distinct test identities, with no dependency on production incidents or real confidential canaries. Access revocation, source replacement, stale vectors, hostile retrieved passages, and evidence-view failures should be exercised with recorded request traces. Provider-backed experiments should then compare declared search and generation conditions on identical eligible cases and retain missing outputs, timeouts, actual model identifiers, and costs where available.

Only after these foundations are established should the analyst study proceed. Institutional approval, informed consent, representative recruitment, and a prospective sample-size justification are prerequisites. The study should test whether evidence inspection and generated explanations change actual decisions, not just preferences for an interface. Confidence calibration and acceptance of unsupported advice should be retained as outcomes alongside correctness and elapsed time.

The existing optional reranker should first be assessed against its disabled condition with a declared candidate pool and configuration. Graph retrieval, semantic chunking, and model fine-tuning are possible later extensions, not missing prerequisites for describing the current artefact. Each should address a measured limitation. Introducing them simultaneously would make causal attribution difficult; the next architecture should be no more complex than the next defensible experiment requires.

### 5.5. Closing statement

The present work positions RAG as a testable knowledge-recovery component inside a broader governed system. Its strongest current contribution is the explicit treatment of authority, provenance, revision, and evaluation boundaries. Whether that design improves the work of cybersecurity analysts remains an empirical obligation. Keeping that distinction clear makes the dissertation a foundation for a credible study rather than a retrospective claim of success unsupported by results.

APPENDIX A

# Implementation Evidence and Reproducibility Boundaries

## A.1. How the evidence should be read

Chapter 3 attributes the implemented governance and evidence mechanisms to the inspected committed `main` revision, while identifying subsequent interface and prompt changes in the local working tree. Chapter 4 reports checks executed against that modified working tree. The return-to-requirements matrix in Table 4.2 uses F1–F4 for functional requirements and N1–N3 for non-functional requirements corresponding to Chapter 3, Table 3.2. In particular, N1 isolates runtime disclosure safety from F1's knowledge-governance workflow. A source definition establishes an available path, a passing isolated test establishes its specified assertion for selected inputs, and an intercepted browser response establishes presentation under a mock contract. None is an expert judgment of security advice or an end-to-end production demonstration.

## A.2. Worked design scenario: publication, citation, and withdrawal

Consider a *proposed case*, not a recorded participant task: an editor submits a revised mitigation playbook; a different, authorised administrator reviews and publishes it. The current revision becomes eligible for asynchronous indexing. An analyst who may read the article asks a question, receives a passage-linked advisory answer, and opens the cited passage. Later the publication expires or the analyst's membership is revoked. The retained citation remains a historical record, while a subsequent passage request should become unavailable under the current policy. A trusted feed may follow a distinct initial-publication policy; it must not be mistaken for human approval in this scenario.

The case crosses several boundaries that a single screenshot cannot exercise. Publication state and revision identity must survive indexing delays; vector matches must be checked against the revision and the reader's rights; the model's prose must be reviewed for actual support; and passage viewing must recheck current eligibility. Table A.1 links these observations back to the named requirements rather than to a source-file inventory.

Table A.1: Design-scenario checkpoints and their evidential status; these are not outcomes of an executed end-to-end case.

| Requirement ID | Mechanism in the proposed case | What would have to be observed |
|---|---|---|
| F1 Governed knowledge | Distinct proposer and approver; active membership, approved publication, and expiry controls. | Record allowed and denied transitions for separate identities, including later revocation; a mocked approval screen would not suffice. |
| F2 Recoverable ingestion | Retained upload bytes or source text, leased processing, and fenced index replacement. | Inject an interrupted worker and verify eventual eligible indexing without stale replacement or fabricated source content. |
| F3 Relevant retrieval | Policy-filtered lexical and vector candidates tied to current evidence. | Freeze the corpus and access policy; inspect ranked, authorised passages against independent relevance judgments. |
| F4 Traceable answers | Revision and passage identity carried into the answer and checked again by the evidence viewer. | Compare each material claim with the actual passage, then attempt the same citation after expiry or access loss. |
| N1 Access and disclosure safety | Read-policy checks at retrieval and response boundaries and safe unavailable views. | Use distinct real-state identities and adversarial content; inspect prompts, responses and logs for unauthorised disclosure. |
| N2 Reviewable operations | Redacted records of consequential publication and access actions, with best-effort writes. | Reconcile expected events with persisted events and disclose any audit-write losses rather than presuming completeness. |
| N3 Deployment flexibility | Independent identity, storage, and inference configuration. | Document the actual hosting topology and approved provider/data-flow settings for each trial. |

### A.3. Evidence identity and verification limits

An article citation conceptually carries the article identifier, revision identity and number, evidence-snapshot identity, and nonempty half-open passage offsets, with source version

and URI when available. The offsets refer to the retained evidence representation, which need not be identical to original upstream bytes. A historical reference does not confer continuing read access; current policy and publication state are rechecked at passage view. Nor does structural identity prove that the answer's claim follows from the passage.

The 29 September 2026 modified-tree verification comprised a twelve-test synthetic-report gate; 408 Java tests in 88 reports without failures, errors or skips; 91 frontend unit tests in 13 files; 17 Chromium tests; and a successful 133-module frontend build. Isolated PostgreSQL containers exercised selected backend paths, while at least the illustrative browser capture used intercepted authentication and application responses. The report validator had also passed an earlier audit, which is not added to the current run. The external implementation and evaluation audits hold exact source locations, while execution records and a source manifest identify commands, environments, hashes, and snapshot scope. These records support reproducibility of bounded engineering claims, not source authenticity, complete audit capture, model quality, or a human-participant dataset. The full-document build and citation checks are a further, separate editorial verification.

APPENDIX B

# Source Evidence and Bibliographic Method

## B.1. Research inputs and limits of inspection

The connected research notes and supplied reference list informed the literature map, but they are not a newly executed systematic search or a completed full-text quality appraisal. A companion reference catalogue preserves each supplied identifier, resolved bibliographic metadata, retrieval status, and evidence level. Unresolved records are not assigned invented metadata or used for substantive claims. The bibliography printed after these appendices contains the works actually cited in the dissertation; stable internal citation keys maintain consistency across chapters without determining printed reference numbers.

Table B.1 illustrates the distinction between a work's reported subject, the material inspected for this thesis, and a conclusion not warranted by that material. External source records are kept in the companion evidence material rather than reproduced as an internal file list here.

## B.2. Claim-to-source examples

Table B.1: Illustrative claim-to-source boundaries in the literature argument.

| Claim or use | Source and evidence available |
|---|---|
| RAG evaluation should distinguish retrieved context, answer faithfulness, and answer relevance. | RAGAs and ARES, with selected methods inspected directly; neither validates this project's outputs.[6, 18] |
| DSRM supplies six activities for artefact-oriented research. | Peffers et al., directly retrieved abstract; adoption of the framework does not complete each activity automatically.[17] |
| Some cyber tasks do not benefit from the tested RAG configuration. | PyPI detection, ForensicLLM, and RAGIntel abstracts; descriptions remain task-specific author-reported results.[2, 8, 19] |

| Claim or use | Source and evidence available |
|---|---|
| Graph construction can introduce noise and conflict. | Li et al., retrieved abstract, not a replicated quality benchmark.[11] |
| Knowledge sharing has organisational dimensions beyond retrieval. | Ipe and Yeboah, retrieved abstracts; transfer to Sentinel's users is an explicit design interpretation.[9, 26] |
| Retrieved text can carry indirect instructions. | Greshake et al., retrieved indexed abstract describing the attack surface; no complete defence claim follows.[7] |

A deposited or indexed abstract is not always a usable abstract: bibliographic fragments can be mislabelled by an index. The catalogue therefore distinguishes the presence of an abstract field from an inspected substantive abstract. Where the text is deficient, the record must be downgraded or replaced before supporting a substantive claim. Direct inspection of selected full-text sections is likewise not equivalent to a complete risk-of-bias appraisal.

APPENDIX C

# Proposed Review and Analyst-Study Instruments

These instruments are proposals for expert refinement. They are not responses collected from participants, validated scales, or evidence of ethics approval.

## C.1. Scenario selection and requirement coverage

The existing answer dataset has nine synthetic cases, all with pending expert review. Table C.1 groups their intended challenges and identifies an additional lifecycle case that must be designed before an end-to-end trial. The first five rows describe developer-authored fixture categories, not observed answer performance. Expected phrases and forbidden identifiers cannot substitute for independent passage judgments or actual multi-identity access tests.

Table C.1: Reader-facing scenario prompts mapped to the requirement identifiers in Table 4.2; no expert or analyst outcomes are reported.

| Scenario group | Requirement IDs | Prospective review question |
|---|---|---|
| Answerable and temporal disclosure (two cases) | F4 | Does each claim follow from the right revision and disclose current versus historical status? |
| Unanswerable and ambiguous (two cases) | F3, F4 | Is absence of authorised evidence distinguished from a question that requires clarification? |
| Conflicting and stale (two cases) | F1, F4 | Are conflicting claims qualified, and is historical scope respected instead of mechanically forcing abstention? |
| Multi-document (one case) | F3, F4 | Are multiple independent, readable passages needed and actually cited for the material claims? |
| Adversarial and access-denied (two cases) | N1, N2 | Does untrusted text remain data and does a second identity fail to obtain protected content, including through traces? |
| Proposed lifecycle/recovery extension (not in seed dataset) | F1, F2, N1 | After failed processing or revocation, is stale evidence neither served nor silently substituted? |

### C.2. Case review form

For each case, record the case identifier and scenario group, intended user role, workspace, frozen corpus and policy versions, question, requested temporal scope, answerability classification, and approved supporting passages. Each passage should carry its immutable identity and eligibility for that identity at the time of the task. Reviewers should specify both essential claims and unacceptable interpretations, including omissions that would make an otherwise true statement misleading. Record a different relevance set when a second identity has different permissions; do not reuse a globally visible answer key for a restricted subject.

The review should answer the following questions independently before adjudication:

1. Is the question sufficiently specified for the intended user and task?
2. Which authorised passages are relevant, and which claims do they actually support?
3. Is a current answer, a historical answer, a clarification, or an abstention appropriate?
4. Are there conflicts, superseded sources, or scope restrictions that must be disclosed?
5. What recommendation would constitute a severe error in this task?
6. Does the case depend on sensitive information that cannot be sent to a proposed provider?

Record reviewer identity, qualifications, review date, rationale, disagreements, and adjudication separately from the answer scores. The stale seed case currently demands abstention even though it asks about a superseded snapshot; reviewers must decide whether a qualified historical answer is instead legitimate before freezing the rubric. Model-generated labels may assist preparation but do not replace independent expert review.

### C.3. Answer review form

For each baseline–case output, retain the actual model and retrieval configuration, eligible corpus and identity, output content digest, failures, and cost/latency observations when available. Segment material claims, associate their citations, and score support, citation entailment, completeness, temporal scope, abstention, and harmful advice. For an apparent disclosure, inspect retrieval, prompt, output, and evidence-view traces rather than the final answer alone. Include an "uncertain" judgement that can trigger adjudication instead of forcing an unsupported binary answer. Record missing output as missing output and preserve its reason; do not silently drop failed cases from a comparison.

The two independent reviews should be made without access to each other's labels. Record pre-adjudication agreement, then a separately identified adjudicator's resolution and rationale. A structurally complete review may still find that the answer was wrong or unsafe. The current fixture has no completed independent annotation.

### C.4. Analyst task record

A proposed record contains a pseudonymous participant identifier, experience stratum, case and condition, allocation order, start and end timestamps, final decision, pre- and post-task confidence, evidence opened, overrides, and acceptance of unsupported advice. Outcome reviewers record decision correctness and error severity separately from perceived usefulness, blinded to condition where feasible. The assigned-case denominator includes timeout, withdrawal, and infrastructure failures with prespecified missingness categories. Actual participants, recruitment figures, consent dates, and responses must be supplied only after the approved study occurs.

Potential post-task prompts include: "Which evidence most influenced your decision?"; "Was any important information missing?"; and "Did you accept a recommendation you could not verify?" These prompts are exploratory, not a validated psychometric instrument. The final study should predefine scoring, counterbalanced allocation, missing-data handling, severe-error stopping criteria, retention, and access to participant records. The instruments here are templates for that approved future work, not a claim that the study or its ethics review has begun.

# References


[1] Chon Abraham, France Bélanger, and Sally Daultrey. Promoting research on cyber threat intelligence sharing in ecosystems. *Journal of Cybersecurity*, 11(1), 2025. doi: 10.1093/cybsec/tyaf016. URL https://doi.org/10.1093/cybsec/tyaf016.

[2] Abeer Alhuzali. LLM-powered threat intelligence: a retrieval-augmented generation approach for cyber attack investigation. *PeerJ Computer Science*, 11:e3371, 2025. doi: 10.7717/peerj-cs.3371. URL https://doi.org/10.7717/peerj-cs.3371.

[3] Nolwenn Bernard and Krisztian Balog. A Systematic Review of Fairness, Accountability, Transparency, and Ethics in Information Retrieval. *ACM Computing Surveys*, 57(6):1–29, 2025. doi: 10.1145/3637211. URL https://doi.org/10.1145/3637211.

[4] Charalampos Bratsas, Efstathios Konstantinos Anastasiadis, Alexandros K. Angelidis, Lazaros Ioannidis, Rigas Kotsakis, and Stefanos Ougiaroglou. Knowledge Graphs and Semantic Web Tools in Cyber Threat Intelligence: A Systematic Literature Review. *Journal of Cybersecurity and Privacy*, 4(3):518–545, 2024. doi: 10.3390/jcp4030025. URL https://doi.org/10.3390/jcp4030025.

[5] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking Large Language Models in Retrieval-Augmented Generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17754–17762, 2024. doi: 10.1609/aaai.v38i16.29728. URL https://doi.org/10.1609/aaai.v38i16.29728.

[6] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.eacl-demo.16. URL https://doi.org/10.18653/v1/2024.eacl-demo.16.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985. URL https://doi.org/10.1145/3605764.3623985.

[8] Motunrayo Osatohanmen Ibiyo, Thinakone Louangdy, Phuong T. Nguyen, Claudio Di Sipio, and Davide Di Ruscio. Detecting Malicious Source Code in PyPI Packages

with LLMs: Does RAG Come in Handy. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 653–658. ACM, 2025. doi: 10.1145/3756681.3757042. URL https://doi.org/10.1145/3756681.3757042.

[9] Minu Ipe. Knowledge Sharing in Organizations: A Conceptual Framework. *Human Resource Development Review*, 2(4):337–359, 2003. doi: 10.1177/1534484303257985. URL https://doi.org/10.1177/1534484303257985.

[10] Anna Kougioumtzidou, Angelos Papoutsis, Dimitrios Kavallieros, Thanassis Mavropoulos, Theodora Tsikrika, Stefanos Vrochidis, and Ioannis Kompatsiaris. An End-to-End Framework for Cybersecurity Taxonomy and Ontology Generation and Updating. In *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 247–254. IEEE, 2024. doi: 10.1109/CSR61664.2024.10679346. URL https://doi.org/10.1109/CSR61664.2024.10679346.

[11] Hongyi Li, Ze Shi, Chengwei Pan, Di Zhao, and Nan Sun. Cybersecurity knowledge graphs construction and quality assessment. *Complex & Intelligent Systems*, 10(1): 1201–1217, 2024. doi: 10.1007/s40747-023-01205-1. URL https://doi.org/10.1007/s40747-023-01205-1.

[12] Fatma Yasmine Loumachi, Mohamed Chahine Ghanem, and Mohamed Amine Ferrag. Advancing Cyber Incident Timeline Analysis Through Retrieval-Augmented Generation and Large Language Models. *Computers*, 14(2):67, 2025. doi: 10.3390/computers14020067. URL https://doi.org/10.3390/computers14020067.

[13] Vasileios Mavroeidis and Siri Bromander. Cyber Threat Intelligence Model: An Evaluation of Taxonomies, Sharing Standards, and Ontologies within Cyber Threat Intelligence. In *2017 European Intelligence and Security Informatics Conference (EISIC)*, pages 91–98. IEEE, 2017. doi: 10.1109/EISIC.2017.20. URL https://doi.org/10.1109/EISIC.2017.20.

[14] Abdullah Mudzakir, Jip Tyrone Ativiirya Janaprasetya, Emanuel Kayowuan, Ventje Jeremias Lewi Engel, and Bagaskoro Saputro. Evaluating Three Large Language Models for Security Incident Detection and Analysis. *Procedia Computer Science*, 269:465–473, 2025. doi: 10.1016/j.procs.2025.08.299. URL https://doi.org/10.1016/j.procs.2025.08.299.

[15] Mohamad Amin Osman, Shahrul Azman Mohd Noah, and Saidah Saad. Ontology-Based Knowledge Management Tools for Knowledge Sharing in Organization—A Review. *IEEE Access*, 10:43267–43283, 2022. doi: 10.1109/ACCESS.2022.3163758. URL https://doi.org/10.1109/ACCESS.2022.3163758.

[16] Jakub Pastuszuk, Patryk Burek, and Bogdan Księżopolski. Cybersecurity Ontology for Dynamic Analysis of IT Systems. *Procedia Computer Science*, 192:1011–1020,

2021. doi: 10.1016/j.procs.2021.08.104. URL https://doi.org/10.1016/j.procs.2021.08.104.

[17] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3):45–77, 2007. doi: 10.2753/MIS0742-1222240302. URL https://doi.org/10.2753/MIS0742-1222240302.

[18] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 338–354. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.naacl-long.20. URL https://doi.org/10.18653/v1/2024.naacl-long.20.

[19] Binaya Sharma, James Ghawaly, Kyle McCleary, Andrew M. Webb, and Ibrahim Baggili. ForensicLLM: A local large language model for digital forensics. *Forensic Science International: Digital Investigation*, 52:301872, 2025. doi: 10.1016/j.fsidi.2025.301872. URL https://doi.org/10.1016/j.fsidi.2025.301872.

[20] Leslie F. Sikos. Cybersecurity knowledge graphs. *Knowledge and Information Systems*, 65(9):3511–3531, 2023. doi: 10.1007/s10115-023-01860-3. URL https://doi.org/10.1007/s10115-023-01860-3.

[21] Marco Simoni, Andrea Saracino, Vinod P, and Mauro Conti. MoRSE: Bridging the Gap in Cybersecurity Expertise with Retrieval Augmented Generation. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, pages 1213–1222. ACM, 2025. doi: 10.1145/3672608.3707898. URL https://doi.org/10.1145/3672608.3707898.

[22] Takeshi Takahashi and Youki Kadobayashi. Reference Ontology for Cybersecurity Operational Information. *The Computer Journal*, 58(10):2297–2312, 2015. doi: 10.1093/comjnl/bxu101. URL https://doi.org/10.1093/comjnl/bxu101.

[23] Thomas D. Wagner, Khaled Mahbub, Esther Palomar, and Ali E. Abdallah. Cyber threat intelligence sharing: Survey and research directions. *Computers & Security*, 87:101589, 2019. doi: 10.1016/j.cose.2019.101589. URL https://doi.org/10.1016/j.cose.2019.101589.

[24] Peng Wang, Jingju Liu, Dongdong Hou, and Shicheng Zhou. A Cybersecurity Knowledge Graph Completion Method Based on Ensemble Learning and Adversarial Training. *Applied Sciences*, 12(24):12947, 2022. doi: 10.3390/app122412947. URL https://doi.org/10.3390/app122412947.

[25] Pu Wang, Yangsen Zhang, Zicheng Zhou, and Yuqi Wang. SC-LKM: A Semantic Chunking and Large Language Model-Based Cybersecurity Knowledge Graph Construction Method. *Electronics*, 14(14):2878, 2025. doi: 10.3390/electronics14142878. URL https://doi.org/10.3390/electronics14142878.

[26] Asiamah Yeboah. Knowledge sharing in organization: A systematic review. *Cogent Business & Management*, 10(1), 2023. doi: 10.1080/23311975.2023.2195027. URL https://doi.org/10.1080/23311975.2023.2195027.